Neural Compression for Multimodal Vehicle Data

Investigating different Architectures for Neural Compression Models for Multimodal Time Series Data

Master's thesis in Computer science and engineering

Tim Boleslawsky
Emrik Dunvald

Department of Computer Science and Engineering
CHALMERS UNIVERSITY OF TECHNOLOGY
UNIVERSITY OF GOTHENBURG
Gothenburg, Sweden 2026

Master's thesis 2026

Neural Compression for Multimodal Vehicle Data

Investigating different Architectures for Neural Compression Models for Multimodal Time Series Data

Tim Boleslawsky
Emrik Dunvald

UNIVERSITY OF GOTHENBURG

CHALMERS
UNIVERSITY OF TECHNOLOGY

Department of Computer Science and Engineering
Chalmers University of Technology
University of Gothenburg
Gothenburg, Sweden 2026

Neural compression for multimodal Vehicle Data
Investigating different Architectures for Neural Compression Models for Multimodal Time Series Data
Tim Boleslawsky
Emrik Dunvald

© Tim Boleslawsky, Emrik Dunvald, 2026.

Supervisor: Yinan Yu, Department of Computer Science and Engineering
Advisor: Dhasarathy Parthasarathy, Volvo Group
Examiner: Peter Damaschke, Department of Computer Science and Engineering

Master's Thesis 2026
Department of Computer Science and Engineering
Chalmers University of Technology and University of Gothenburg
SE-412 96 Gothenburg
Telephone +46 31 772 1000

Typeset in LaTeX
Gothenburg, Sweden 2026

Neural compression for multimodal Vehicle Data
Investigating different Architectures for Neural Compression Models for Multimodal Time Series Data
Tim Boleslawsky
Emrik Dunvald
Department of Computer Science and Engineering
Chalmers University of Technology and University of Gothenburg

Abstract

With the growing amount of data generated by modern vehicles, the need for edge efficient data processing and compression strategies becomes more apparent. As the amount of data grows so does the potential for machine learning models to extract insightful information from this data at later stages. Achieving high compression ratios while retaining only the essential components of the original data therefore becomes a the main goal of many compression strategies. Currently, the existing solutions for such a task on a resource constrained devices such as a vehicle are limited and often come with tradeoffs. This thesis investigates compression algorithms based on neural networks (commonly referred to as neural compression) which have shown promising results in other fields, such as image processing. Four neural codec architectures are explored and tested on industry relevant datasets with compression ratio, data reconstruction quality and downstream utility retention as the main metrics investigated.

Keywords: Signal Data, Time Series, Multimodal Data,Neural Compression, Edge Computing, Neural Codec, Autoencoder, Vector Quantization.

Acknowledgements

First, we would like to thank our supervisor, Yinan Yu, for her guidance and support throughout this project. Her feedback and insights were valuable for designing the thesis and overcoming challenges along the way.

We would also like to thank the team at Volvo Group for providing technical support, access to their infrastructure and for their great support. In particular, we would like to thank our company advisor, Dhasarathy Parthasarathy, for his support and feedback on the project. We would also like to thank Szabolcs Torma and Alexander Bergersen for their help and support with technical matters and for providing feedback on the project.

Lastly, Tim would also like to thank his girlfriend Marie for always being there and always showing support.

Tim Boleslawsky, Gothenburg, 2026-05-08
Emrik Dunvald, Gothenburg, 2026-05-08

Contents

List of Figures

List of Tables

1 Introduction

Modern vehicles generate large amounts of telemetry data through distributed embedded systems, sensors, actuators, and in-vehicle networks [1]–[3]. These systems are designed to support real-time functionality, but they are also constrained by limited communication bandwidth, storage capacity, and on-board compute resources [2]. In practice, this has led to the widespread use of event-triggered and threshold-based logging strategies, which reduce data volume by transmitting only selected events or diagnostic messages [2], [4]. While these approaches are effective for limiting network load, they also risk omitting information that may later be relevant for offline analysis or machine learning-based applications [2], [5].

This challenge has become more important as vehicles generate increasingly rich signal-based data and as downstream machine learning tasks have become a central use of automotive telemetry [6]–[8]. Collected vehicle data is no longer used only for diagnostics or monitoring, but also for tasks such as predictive maintenance, anomaly detection, fleet analytics, and other forms of offline model training [8]–[10]. For these use cases, data utility matters as much as data volume. Aggressive reduction of telemetry may improve efficiency, but it can also degrade the performance of models trained on the resulting data [11]–[13]. This creates a fundamental tension between efficient storage and transmission on the one hand and preserving task-relevant information on the other.

The theoretical basis for this tension comes from information theory and rate-distortion theory [14], [15]. Information entropy describes how much uncertainty is contained in a signal and therefore provides an indication of how compressible that signal is [14]. Rate-distortion theory extends this idea to lossy compression by formalizing the trade-off between bitrate and reconstruction quality [15]. For machine learning settings, this trade-off is not sufficient on its own, because good reconstruction does not necessarily imply that the compressed representation is useful for downstream tasks [16], [17]. This is why modern learned compression methods are often framed in terms of autoencoders and information bottlenecks. These methods aim to produce compact latent representations while retaining the information that is relevant for later prediction or classification tasks [18]–[21].

Within this thesis, neural compression is the central approach for addressing this problem. Neural compression methods use learned encoder-decoder architectures to transform high-dimensional input signals into more compact representations. In contrast to traditional algorithmic compression methods, these models can adapt to the structure of the data and can be optimized for the kinds of signals that occur in automotive telemetry [18], [22], [23]. In particular, vector quantization offers a practical way to introduce discrete bottlenecks that reduce entropy while still allowing the model to preserve useful structure in the compressed representation [18], [24]. This makes neural compression especially attractive for edge-oriented settings, where the compressed data must be stored, transmitted, or processed under strict efficiency constraints [24], [25].

Recent research shows that neural compression has been successful in a range of domains, including time-series, audio, speech, and other resource-constrained applications [22]–[29]. For time-series data, continuous-latent autoencoder approaches have demonstrated strong reconstruction performance, while vector-quantized methods such as EdgeCodec show that lightweight discrete compression can be effective in edge-deployment scenarios [22], [24]. At the same time, the literature also shows that many methods primarily optimize reconstruction error, whereas downstream task utility is often treated as secondary or is not investigated in depth [22], [24]. For vehicle telemetry, this is a crucial limitation, because it is not shown, if a method that reconstructs well, also performs well on downstream tasks.

The background of this thesis therefore combines three strands of work: the properties and constraints of automotive telemetry, the information-theoretic foundations of compression, and the recent development of neural compression methods. Taken together, these areas motivate the central question of the thesis, namely how to compress multimodal vehicle telemetry in a way that remains useful for downstream machine learning tasks while still achieving meaningful compression and efficiency gains. The following sections build on this background by stating the concrete aim of the thesis, defining the research questions, and outlining the limitations of the study.

1.1 Aim

The aim of this thesis is threefold. First, two existing neural compression methods are applied to automotive and IoT time-series data in order to evaluate how they perform in terms of compression ratio, reconstruction quality, and computational efficiency. One of these methods is based on vector quantization, while the other serves as a continuous-latent baseline.

Second, the compressed representations produced by these methods are used to train downstream machine learning models and are compared against models trained on uncompressed data. This makes it possible to assess how much task-relevant information is retained after compression and reconstruction.

Third, the two methods are further explored through changes to the quantization strategy and loss function. The goal of this exploratory part is to investigate whether reconstruction loss and downstream utility retention can be improved further without sacrificing the general compression objective.

1.2 Research Questions

This thesis is guided by the following research questions:

1. RQ1: How do the selected neural compression methods perform on automotive and IoT time-series data in terms of compression ratio and reconstruction quality?
2. RQ2: To what extent do downstream machine learning models trained on reconstructed compressed data retain task performance compared to models trained on uncompressed data?
3. RQ3: Where do the existing neural compression methods fall short in terms of reconstruction quality and downstream utility retention, and which quantization strategies and loss functions can mitigate these limitations?

1.3 Limitations

Due to time constraints and the scope of this thesis, there are several limitations to the work presented.

First, the work forgoes testing on real hardware as that is very time consuming and difficult to set up while contributing relatively little to the main research question. Instead, canonical methods to measure computational efficiency are employed.

The work is further limited to testing only regression tasks and anomaly detection as downstream tasks due to time constraints and the structure of the available datasets. These two tasks should, however, be a good representation of common downstream tasks, and adding more tasks would likely not drastically change the conclusion of the thesis.

The last noteworthy limitation is that this work does not measure computational efficiency of the evaluated neural compression models. This choice is driven by two practical reasons. First, implementing and benchmarking on representative edge hardware is too complex and time consuming for the scope of this thesis. Second, EdgeCodec's prior work already demonstrates an efficient encoder design, which, importantly, we do not change in the work presented within this thesis. For these reasons, explicit runtime or hardware measurements are considered out of scope for this thesis.

2

Background

This section introduces necessary context for further discussion within this thesis. The in-vehicle system and related components will be introduced, followed by an introduction to modern challenges in regard to data processing and data usage in the automotive industry. The purpose of this chapter is to provide the necessary background for the reader to understand the motivation and relevance of the research presented in this thesis. It also serves to establish a common understanding of key concepts and terminology that will be used throughout the thesis.

2.1 In-Vehicle Systems and Event-Triggered Logging

The In-Vehicle Embedded System An embedded system is a computer system integrated into a physical device or product to perform a dedicated function as part of a larger system [1, pp. vii–x, 1–3]. The in-vehicle embedded system is a specialized embedded system integrated within a vehicle. It is essential for real-time functions like controlling, monitoring, and enhancing various automotive operations [2, pp. 1-1–1-5]. These in-vehicle embedded systems, like embedded systems in general, typically consist of both hardware and software components, such as electronic control units (ECUs), sensors, actuators, and communication interfaces [1, pp. 3–4], [2, pp. 1-3–1-10].

ECUs serve as the primary processing nodes in in-vehicle embedded systems. They are responsible for executing control algorithms on sensor inputs and issuing commands to actuators. They also handle local control logic while coordinating via in-vehicle networks to form a distributed system [2, pp. 1-8–1-19]. An example of how ECUs work together to create the in-vehicle embedded system is illustrated by [2, pp. 1-8–1-9]. In the example an ECU coordinates a request, e.g., locking or unlocking a door, while the controller realize the request on the physical system, which is controlled by another ECU. The resulting system is the main generator of telemetry data within the vehicle. Vehicle telemetry data, usually defined as data that is produced by the vehicle and sent off-board, is key to improving important aspects of these vehicles like safety [3, pp. 1–5]. This vehicle telemetry data is at the core of this thesis.

In-Vehicle Networks As mentioned above, in-vehicle networks serve as the connectors between the ECUs of the in-vehicle embedded system. Among these in-vehicle networks Control Area Networks (CAN), Local Interconnect Networks (LIN), FlexRay, and Ethernet are popular representatives that enable efficient communication and coordination among different vehicle subsystems [2, pp. 1-2–1-5].

From a technical perspective, ECUs connect to these networks via dedicated communication services. The communication services are tasked with providing a uniform interface to the network, while hiding protocol and message properties from the application [30, p. 48]. The in-vehicle network protocols are categorized into three classes, A, B, and C. LIN falls into category A with a bit rate below 10 kbps and application in sensor and actuator networks. CAN-B is a representative of category B with a bit rate between 10–500 kbps. Category B networks serve vehicle-centric domains and body electronics. Category C in-vehicle networks like FlexRay or CAN-C are defined by having a bit rate of lower than 1 Mbps and used in safety-relevant systems [2, pp. 1-12–1-13]. Despite their effectiveness in their respective use-cases, all in-vehicle networks must contend with fundamental bandwidth limitations that constrain data throughput across the vehicle [2, pp. 1-18–1-19].

Event-Triggered and Time-Triggered Logging Event-triggered logging and diagnostic frameworks, which record data only when specific error codes occur or threshold crossings are detected, are widely adopted to reduce data transmission and avoid bus saturation in complex systems such as the in-vehicle embedded system [4, pp. 7–13]. According to the AUTOSAR Diagnostic Log and Trace (DLT) standard, this mechanism can be implemented as follows: When software components or Basic Software modules report errors via the Default Error Tracer, the DLT module receives these via a specified function and stores timestamped log/trace messages to flash storage [4, p. 25]. Non-critical events are filtered during normal operation [4, pp. 30–35].

Time-triggered logging serves as an alternative to event-triggered logging and works by setting predetermined points in time where data is transmitted. The validity of this strategy is easily monitored, as the time-frame scheduling is predictable and missing messages are immediately identified [2, pp. 4-5]. Although Navet discusses these paradigms in the context of communication scheduling, similar principles apply to diagnostic event- and time-based mechanisms, which leads to similar struggles in these systems.

While these approaches have been effective in reducing the data load of the in-vehicle networks, they come with major limitations. Two immediate issues these diagnostic frameworks struggle with is presenting an unbiased and comprehensive monitoring of the system, while having to carefully tune event thresholds and diagnostic parameters [2, pp. 4-5–4-6], [5]. In the next section we will go further into how these limitations are accentuated by recent developments.

2.2 Downstream Machine Learning Tasks and Data Quantity

Two developments in recent years further underline the shortcomings of event-triggered logging in automotive systems: the massive increase in signal-based data in the in-vehicle network and the growing relevance of downstream machine learning (ML) tasks.

Data Quantity in Modern Vehicles Recent industry and research reports indicate that the data quantity generated by sensors and the amount of interconnectivity in vehicles is growing at an extremely rapid pace. According to a 2021 automotive electronics report, by 2030, about 95% of new vehicles will be connected, up from around 50% today [6]. Another report states that even today, a single car can generate up to 1 terabyte (TB) of data per hour from its sensors [7]. This explosive growth is driven by the increasing number and sophistication of sensors—such as cameras, radars, and lidars—required for advanced safety and autonomous driving features, with the complexity and volume of data presenting significant challenges for storage, processing, and transmission within embedded automotive systems [7], [31, pp. 3–5]. While much of the focus in research and reporting on this issue centers on autonomous driving technologies like lidars and cameras, it is important to note that the core of the in-vehicle system, the ECU, is also a major contributor to the data quantity issue. This implies that not only autonomous driving technologies but modern vehicles in general are facing these challenges [31, pp. 3–4].

Downstream ML Tasks Modern vehicles increasingly rely on data-driven intelligence to enhance safety, reliability, and efficiency. As there currently exist no conclusive definition of these tasks, we will refer to them as downstream ML tasks within the context of this thesis. These downstream ML tasks we define as the offline use of collected vehicle data for the training of ML models for various tasks. This is distinctly different from real-time perception and control systems. Downstream ML tasks have become central to automotive-system design. Theissler et al. [8, p. 2] mention three reasons for the rise of importance of these downstream ML tasks: data availability, breakthroughs in algorithm development and the advancements in computational power. Prominent examples of downstream ML tasks in the automotive context include: Predictive maintenance tasks aim to predict the expected time of a failure, thereby helping to estimate remaining functionality of components or systems [8, p. 3]. Anomaly and intrusion detection tasks want to find anomalies or outliers in data points which is especially relevant in fragile systems like the CAN network [9, pp. 1–3]. Fleet-level management operations like fuel consumption that aim to better utilize important assets in vehicle systems [10, pp. 1–2]. Utility loss in the datasets used to train the models for these tasks translates directly to degraded model performance, increased operational costs, and missed business opportunities [31, pp. 4–5]. This further underlines the importance of a comprehensive data logging strategy.

There exists relatively little research on the use of event-triggered logging systems as a compression strategy and it's impact on downstream ML tasks. It is therefore difficult to definitively say that event-triggered logging systems specifically produce data with lower utility. Studies have however shown that similar utility agnostic data reduction strategies can have negative impacts on downstream utility in the domain of time-series data. One such example comes from a study by [11] where the authors show that a 10-minute aggregation of the data leads to degraded utility for downstream ML tasks. In this specific case, the F1-score of the predictive downstream model was reduced by from 0.63 to 0.50 when trained on the aggregated data compared to the original dataset. The same study also shows that a model based lossy compression technique can be implemented without significant utility loss which suggests that more strategic data reduction techniques can retain more utility for downstream ML tasks. Further analysis of the impact of lossy compression algorithms on time series task utility in general has been done by [12] and [13]. It is important to note, that, while challenging, at specific error bounds compression techniques within these studies can actually show improvement to the task metrics. All this highlights the need for a strategic lossy compression technique that accounts not only for reconstruction, but also for downstream task utility.

2.3 Related Work

Time-Series Neural Compression Neural compression for time-series data has been studied, but many methods are not optimized for edge deployment. The architecture proposed by Zheng and Zhang [22] is representative of this line of work. Their method uses a continuous-latent autoencoder with recurrent modules and is primarily optimized for reconstruction quality, while computational efficiency and downstream task utility retention are not central optimization targets [22].

Edge-Optimized Neural Compression The majority of research labeled as “edge-optimized neural compression” focuses on compressing ML models for deployment rather than compressing input data. Lai et al. [32] highlight the importance of enabling complex deep learning models on resource-constrained edge devices. Key developments include improved efficient transformer variants and knowledge-distillation methods [32]. These advances address one side of the edge-optimization problem (model compression), while the complementary side is efficient compression of the data itself. EdgeCodec is a notable example of this latter direction and is discussed below.

Audio/Speech Neural Compression The audio/speech community contains extensive research on neural compression, with a strong emphasis on vector quantization. A key development in this literature is residual vector quantization (RVQ). RVQ works by layering hierarchical quantizers whith each layer quantizing the residuals of the previous layer [27, pp. 495–507].

- SoundStream (Google, 2021): first major use of RVQ for audio compression [27, pp. 495–507].
- EnCodec (Meta, 2022): refined RVQ for high-quality audio [28].
- WaveTokenizer (2024): recent RVQ-based audio codec [29].

EdgeCodec EdgeCodec transfers RVQ ideas from audio/speech processing to time-series compression (aerospace sensor data) and is explicitly designed for edge deployment. This is achieved through a lightweight encoder deployed on the edge and a heavier decoder executed in the cloud [24]. As a result, EdgeCodec is a notable outlier in neural compression literature and addresses many challenges apparent in the research field. However, it has important limitations for the present use case. For one, EdgeCodec is optimized for smooth, continuous aerospace sensor streams rather than mutlimodal vehicle telemetry. And second, EdgeCodec focuses on reconstruction error and omits analysis of downstream task utility retention.

The literature review indicates that neural compression, VQ-based quantization, and lightweight design principles have all been explored individually. The remaining gap is a method tailored to mutlimodal data that explicitly optimizes downstream ML task utility rather than reconstruction quality alone. Based on this gap, the present thesis evaluates two existing solutions, namely the continuous-latent autoencoder by Zheng and Zhang [22] and EdgeCodec [24], with respect to reconstruction quality on multimodal data and downstream ML task performance. Baseline models derived from these works are implemented and evaluated on automotive and IoT telemetry data, with the objective of identifying and improving design choices for multimodal data and downstream utility retention.

3

Theory

This section provides the theoretical background necessary to understand the research presented in this thesis. It covers the fundamental concepts of information theory, compression theory, and neural compression, as well as the specific architectures used as baselines in this work. The goal is to establish a solid theoretical foundation for the subsequent empirical evaluations and proposed improvements.

3.1 Compression Theory and Traditional Methods

Information Theory Because information theory provides the basis for compression methods, its core concepts are introduced here before discussing compression itself. Entropy is key to information theory. It measures the uncertainty of a random variable and is computed from its probability distribution. More specifically, the uncertainty in a probability distribution is the expected negative log-probability of its outcomes. The entropy of a discrete distribution $p = (p_i)_i$ is defined as

$$H(p) = -\sum_i p_i \log p_i,$$

where the base of the logarithm determines the unit [14].

To illustrate with a weather example, suppose the true probabilities of "rain" and "shine" are $p_1 = 0.3$ and $p_2 = 0.7$, respectively. Then (using base-2 logs),

$$H(p) = -(p_1 \log p_1 + p_2 \log p_2) \approx 0.61.$$

Suppose the measurements are instead from Abu Dhabi, where the probabilities might be $p_1 = 0.01$ and $p_2 = 0.99$. The entropy is now approximately 0.06. There is thus much less uncertainty about any given day compared to a place where it rains 30% of the time. In this way, information entropy quantifies the inherent uncertainty in a distribution of events.

Entropy is central to compression because it indicates how compressible data is. High-entropy data is less predictable and therefore harder to compress, while low-entropy data has redundant patterns and is therefore easier to compress [14].

The other important information theory concept to understand in the context of compression is mutual information. Mutual information, formally defined as $I(X;Y)$ answers the question "How much does one variable tell me about another?" [14].

Compression Compression, as originated in information theory by [14], is the process of encoding information using fewer bits than the original representation. Compression techniques can be broadly categorized into lossless and lossy methods. Lossless compression is based on two principles: distribution modeling, sometimes called entropy modeling, and entropy coding. Entropy modeling involves creating a probabilistic representation of the data, while entropy coding assigns shorter codes to more frequent symbols based on their probabilities, thereby minimizing the average code length. Entropy serves as the lower bound (in bits per symbol) for any lossless compression scheme. Lossy compression allows for some loss of information in exchange for higher compression ratios. This is typically achieved through techniques such as transform coding and quantization [33]. Closely connected to lossy compression techniques is the term "compression ratio". compression ratio (CR) is defined as the size of uncompressed data divided by the size of compressed data [34]. For the purpose of this thesis, the focus will be on lossy compression as this is more suitable for common downstream ML tasks where some loss of fidelity is acceptable as long as the relevant information for the task is preserved.

Rate-Distortion Theory Rate-distortion theory provides the fundamental framework for lossy compression, quantifying the minimum bitrate $R(D)$ required to represent data with acceptable distortion at most D. Building on entropy and mutual information introduced earlier, it formalizes the fundamental tradeoff between rate and fidelity central to lossy compression. The rate-distortion function is formally defined as

$$R(D) = \min_{p(\hat{X}|X):\ E[d(X,\hat{X})] \leq D} I(X; \hat{X}),$$

where $d(x, \hat{x})$ is a distortion measure, e.g., squared error, $E[\cdot]$ is the average over all data points or expectation, and the minimum bitrate is computed over all distributions $p(\hat{X}|X)$ achieving average distortion at most D. Shannon's rate-distortion theorem proves this isn't just a theoretical lower bound and practical codes exist that get arbitrarily close to $R(D)$ bits/symbols while keeping distortion below D. [15].

Distortion measures quantify the cost of representation between original data X and its compressed reconstruction $\hat{X}$. They are essential in rate-distortion theory, where the goal is to minimize bitrate subject to $E[d(X, \hat{X})] \leq D$ for a chosen distortion threshold D. Widely used examples for distortion measures include the squared error (MSE) or the Hamming distance. MSE is defined as $d(x, \hat{x}) = (x - \hat{x})^2$, and is often used for numerical/time-series data due to its mathematical tractability. The Hamming distance on the other hand is defined as $d(x, \hat{x}) = \sum_i \mathbf{1}\{x_i \neq \hat{x}_i\}$, and is used for binary or categorical data [35, pp. 304–305].

In the ML context, particularly in automotive systems, or in fact Internet-of-Things (IoT) systems in general, rate-distortion analysis denotes the trade-off between efficiently handling large quantities of data and maximizing model performance. So in this sense, rate-distortion theory gives the theoretical framework to the downstream ML task challenges described in Section 2.2. Traditional lossy compression techniques can reduce data volume, but often at the cost of losing critical information necessary for accurate ML tasks such as predictive maintenance, anomaly detection, and fleet analytics [16]. The impact of this trade-off is well-documented in the literature. Cuza et al. [17] for example, study the impact of lossy compression techniques on time series forecasting tasks and observe a constant trade-off between CR and forecasting accuracy. It is important to note, that, while the authors of this paper see impact of CR on task performance, it is not necessarily a negative impact and is dependent on various factors [17, pp. 654–655].

Traditional Compression Methods Traditional compression methods, based on information theory principles, often fall short in automotive applications, especially as a precursor for downstream ML tasks. For video/image compression, traditional methods like JPEG are optimized for human perception (e.g., visual quality) rather than ML tasks or efficient downstream data use [36, p. 12]. For time series data, algorithmic approaches like CHIMP or Gorilla depend on manually chosen parameters like window size and are sensitive to data characteristics such as entropy and signal variability. This limits their effectiveness in capturing the nuances required for accurate ML model performance in automotive contexts [37, pp. 37–38 and 47]. These algorithmic approaches were investigated by Johnsson [37] in a previous Master's thesis project. This work builds upon this thesis by exploring neural alternatives for vehicle telemetry compression. These neural alternatives are more flexible and can be optimized for specific downstream tasks and data characteristics, potentially offering better performance in terms of the rate-distortion trade-off for ML tasks. As previously shown by [11], model aware compression techniques have the potential to retain more task relevant information compared to traditional methods, which is key for achieving better performance for downstream ML tasks while still maintaining significant CR.

3.2 Neural Compression

In the most basic sense, neural, or sometimes learned compression, is the application of neural networks and related machine learning techniques to the task of data compression [18]. Neural compression has been shown to be an effective strategy for handling the data rate-utility trade-off in various domains. Studies as early as 2019 have shown that neural compression methods can outperform traditional compression techniques for image and video data, especially at low bitrates [26]. The same has been shown for time series data [22], [23]. This approach has also been shown to be effective in resource-constrained environments, such as IoT systems, where the ability to retain task-relevant information while reducing data volume is crucial [24], [25].

In the following sections, the basics of neural compression theory and the connection to relevant information theory concepts are outlined.

Information Bottleneck Theory While the theory presented by [14] gives a theoretical framework for understanding the limits of compression, it falls short when it comes to learned compression as it only considers the maximum CR for a given distortion level, without considering the specific characteristics of the data or the downstream tasks. This shortcoming is addressed in the information bottleneck (IB) theory, which presents a more nuanced approach to the issue of compression. In order to maintain relevance for downstream tasks, we need to preserve as much relevant information as possible about the target label, not about the input data. This is the core idea of the IB theory, which formalizes the trade-off between compression and relevance as:

$$\min_{p(\hat{X}|X)} I(X;\hat{X}) - \beta I(Y;\hat{X}),$$

where $I(X;\hat{X})$ is the mutual information between the input data X and its compressed representation $\hat{X}$, $I(Y;\hat{X})$ is the mutual information between the target label Y and the compressed representation $\hat{X}$, and β is a Lagrange multiplier that controls the trade-off between compression and relevance [19].

Early work on attempting to understand the learning dynamics of deep neural networks (DNNs) have applied the IB theory in an attempt to explain how DNNs learn compressed representations of data while preserving relevant information for specific tasks [38, pp. 1-5]. The core of this idea is that the goal of any supervised learning algorithm is to capture a minimally sufficient statistic in the input required to express the output. This mirrors the goal of IB compression which claims that there exists a maximally compressed representation of the input which retains the maximum amount of information about the target label. Other works such as [39] have attempted to show that DNNs both fit the input in order to increase the mutual information between the target and the latent representation $I(T;Y)$ while also compressing the input by lowering the mutual information between the input and the latent representation $I(X;T)$. This work has however been criticized for its methodology and computation of mutual information. Much of the issue comes down to the fact that the layers of a DNN cannot lower mutual information, but rather just learn to map the input to consistent output values, effectively decorrelating the data structure. This process will be referred to as geometric compression. In order for a network to be able to discard information and thereby compress the input in an IB sense, additional measures are required [40], [41]. So how DNNs learn is debated. But it becomes clear that geometric compression, while being dependent on the architecture or the used activation functions, is key to how DNNs function. Modern neural compression research therefore almost always combines geometric compression supplemented by some form of information-theoretic compression.

Autoencoders & Vector Quantization The implementation of lossy neural compression methods is firmly based on autoencoders (AEs) and more specifically on Rate-Distortion Variational Autoencoders or R-D VAEs [18, pp. 145–146]. Generally AEs are defined as DNNs that try to reconstruct their input. This implies that the target output is the input. For this, AEs are composed of an encoder, which produces a hidden or latent representation, and a decoder, which takes this representation as an input and tries to reconstruct the original input. AEs are trained end-to-end, which means that the encoder and the decoder are trained in tandem. The training goal is to make the latent representation take on meaningful properties [20].

The main difference between AEs and VAEs, introduced by Kingma and Welling [42], is that VAEs pick from a distribution of points when assigning latent representations, while AEs are deterministic in nature [20]. Importantly, many modern learned compression methods are formulated in a VAE-style rate-distortion framework. This is because an AE is essentially a VAE with a degenerate (Delta) variational posterior distribution. A deterministic autoencoder maps x to a single point $\hat{z} = f(x)$; the variational posterior $q(z|x)$ then becomes a Delta distribution and therefore deterministic [18, pp. 145–146]. The choice between using a stochastic encoder or a deterministic encoder with quantization is an ongoing debate in the research community. Because of the non-differentiability of quantization, and the resulting problems in end-to-end training of lossy compression models, there has been experimentation with stochastic encoders. Theis and Agustsson [43] summarize the advantages of stochastic encoders, and while there certainly are advantages to stochastic encoders, deterministic encoders are favored by most modern neural compression methods [18], [43]. No matter which of these approaches are implemented, they both display the compromise between geometric compression and information-theoretic compression outlined above. In the case of deterministic encoders, which are the focus of this thesis, the information-theoretic compression is achieved via quantization, which will be introduced in more detail in the next paragraph.

Quantization is often used in neural compression to introduce true information-theoretic compression when using deterministic encoders. While there have been various approaches to achieving true compression of information in the context of IB using neural networks, such as the variational information bottleneck (VIB) architecture [21], the approach that is presented in this thesis extends the traditional autoencoder architecture with a VQ bottleneck in order to minimize the entropy. This idea is conceptually more similar to methods such as those presented in [44, pp. 1611–1630]. The VQ layer acts as a deterministic information bottleneck partitioning the input space X into regions that map to the same discrete code t, discarding intra-cluster variations. By binning the continuous signal to a finite set of codes, the entropy of the signal is bounded by the number of bins. As mentioned in [40], the encoder cannot exhibit decreasing $I(X;T)$ without noise or stochasticity since the entropy is constant $H(T \mid X)$. It can however create sharper decision boundaries which allow the VQ bottleneck to more effectively quantize the input without discarding relevant information.

In neural compression methods, autoencoder architectures are typically chosen based on the structure of the input signals, the computational restraints, and the target rate-distortion objective. For telemetry and other mutlimodal time series data convolutional neural networks (CNNs) and recurrent neural networks (RNNs) are popular design choices [18, pp.146–148]. CNNs naturally excel at capturing local temporal patterns efficiently, while RNNs are used to model longer-range dependencies [18, pp.146–148]. Both CNNs and RNNs are especially suitable for neural compression, because they use a more compact parametric representation than for example fully connected neural networks [45]. Naturally, a lot of hybrid systems trying to combine CNNs and RNNs have also emerged over the years. In practice, this architectural choice is not only a modeling decision but also a systems decision. Lightweight designs usually avoid costly recurrent designs and prefer convolutional approaches when compression is deployed on resource-constrained edge devices with strict latency and memory budgets [24], [46].

3.3 Baseline Implementations

In this section, the architectures of the established compression methods which are used as baselines are introduced in detail. If not stated otherwise in this section, all configurations for the models are identical to those of the respective authors. A detailed config file for each specific experiment is provided in the next chapter.

3.3.1 Baseline 1: TCN-RNN

The continuous-latent autoencoder approach introduced by Zheng and Zhang [22] serves as the basis for the continuous-latent baseline model. Their work includes two models, the TCN-RNN model and the TCN-ARNN model, as well as a model selection mechanism. For simplicity, the focus here is on the TCN-RNN model. The TCN-RNN model is representative of state-of-the-art continuous-latent neural time-series compression methods. Compared to vanilla autoencoders, its architecture is explicitly designed to capture long temporal dependencies and has demonstrated stronger rate-distortion performance. This makes it a suitable baseline for evaluating the proposed implementation against a competitive neural compressor. Another point to note about the TCN-RNN model is its costly architectural design, namely deep temporal convolutional stacks with large receptive fields, recurrent (often sequential) decoding, and dense continuous latent representations. These components typically lead to increased FLOPs, higher activation memory, and longer inference latency.

The TCN-RNN model is composed of four main components: the TCN-RNN encoder, a linear projection that serves as the bottleneck, the GRU decoder, and a second linear projection. A complete overview of the TCN-RNN architecture is shown in Figure 3.1.

The TCN encoder consists of a stack of temporal convolutional network (TCN) residual blocks with increasing dilation factors. Each TCN block is primarily composed of two causal convolutional layers, with normalization, a nonlinear activation function (ReLU), and optional dropout for regularization. Additionally, each TCN residual block implements a residual connection to stabilize training. The output is a continuous latent representation whose size is controlled by the downsampling factor and channel width. As in Zheng and Zhang [22], there is no downsampling in the decoder; any downsampling occurs in the linear bottleneck, which is described in the next paragraph. As a result, the input shape of the encoder is equal to its output shape. The detailed TCN encoder architecture is shown in Figure 3.2a. In addition to common design choices such as dropout, residual connections, and the use of ReLU non-linearity, this encoder includes several noteworthy elements. It is designed with training efficiency and information preservation in mind. To this end, the authors use exponential dilations and additional layers to achieve stronger long-range temporal modeling. This design intentionally trades some computational efficiency for modeling capacity. The authors also employ weight normalization to improve training stability, which further increases computational cost.

After the TCN encoder, the linear bottleneck provides the only source of geometric compression within the TCN-RNN architecture. This layer is a linear projection that

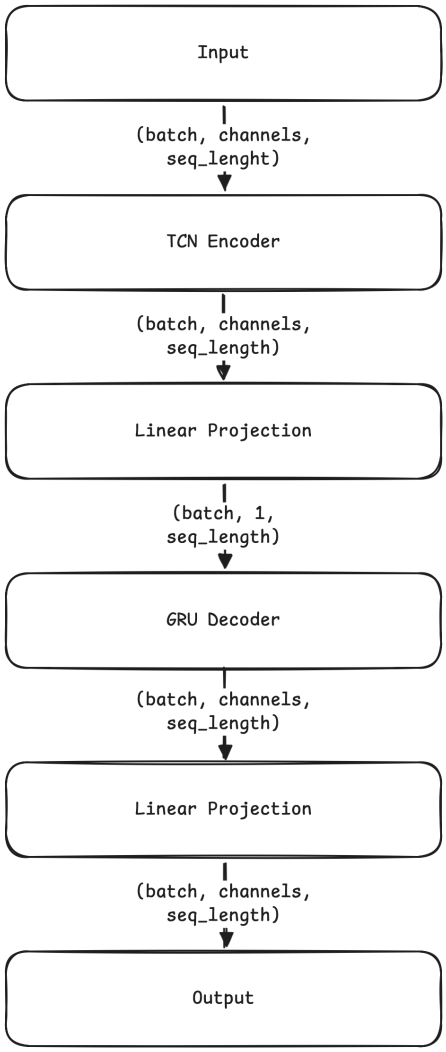
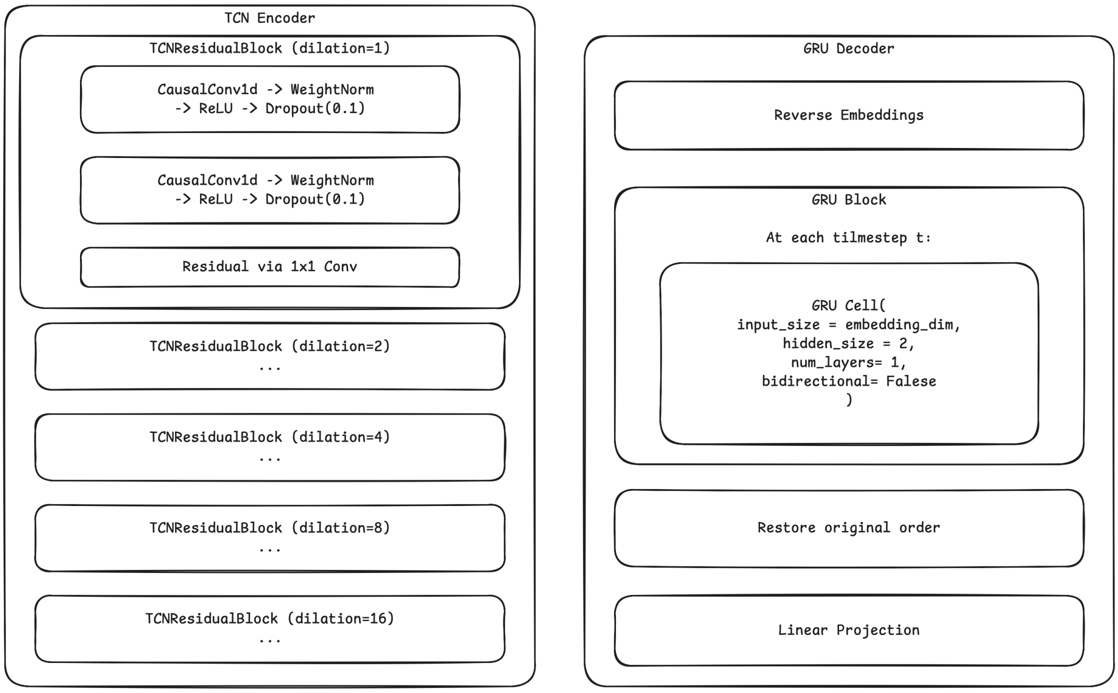


Figure 3.1: TCN-RNN architecture

reduces the input channel dimension to one. In the case of Zheng and Zhang [22], two input channels were used, resulting in a CR of 2:1. This means that the linear bottleneck layer of the TCN-RNN architecture produces a latent representation of the form $(\text{batch}, 1, \text{seq\_length})$.

The continuous latent representation produced by the linear bottleneck serves as the input to the decoder. The decoder has two main functions. First, the embeddings are reversed for reconstruction in reverse order, meaning that the sequence is processed from end to beginning. The authors use this strategy to help the model focus on short-range temporal correlations. Second, the decoder uses a gated recurrent unit (GRU) to reconstruct the input. GRUs are a variant of RNNs that introduce reset and update gates to control the hidden state. This GRU cell expands the latent representation compressed by the bottleneck back to its original dimensionality. Afterwards, the original order is restored and a final linear projection is applied. Figure 3.2b shows this decoder setup in more detail.

The loss function used for the TCN-RNN baseline is identical to the loss function



(a) TCN encoder (b) GRU decoder

Figure 3.2: Detailed TCN-RNN components

outlined by Zheng and Zhang [22]. They use a simple MSE loss, but with the important caveat that they use the sum-reduced variant of MSE. The following equation shows the exact loss formula used:

$$L_{\text{MSE}} = \frac{1}{T} \sum_{t=1}^{T} \sum_{d=1}^{D} \left(x_t^d - \hat{x}_t^d \right)^2 \tag{3.1}$$

This function is used as the loss function for the TCN-RNN model, while the mean-reduced version of MSE is used to evaluate all the models, as described earlier in this chapter.

To validate the correctness of this implementation, a small use case from the paper is reporduced. In the paper Zheng and Zhang [22] use the appliances energy dataset, which will be introduced at a later stage, to compress and reconstruct the two signals "Windspeed" and "Appliances". The authors present a summed MSE reconstruction error of 0.178. The implemented TCN-RNN baseline achieves a summed MSE reconstruction error of 0.133 with the same configurations, showing that it works as intended.

3.3.2 Baseline 2: EdgeCodec

The EdgeCodec implementation leverages residual vector quantization (RVQ) and a heavyweight decoder to enable a lightweight encoder architecture, which is edge-optimized [24]. The compression mechanism of the EdgeCodec architecture is fundamentally different from the linear bottleneck employed by the TCN-RNN model. The compression achieved by EdgeCodec is a mixture of geometric compression in the encoder and compression in the information theoretic sense through quantization. The EdgeCodec encoder compresses both along the channel and the temporal axis and works as a dimensionality reduction mechanism. The quantization mechanism employed by EdgeCodec is a 4-layer hierarchical RVQ setup. The full EdgeCodec architecture is displayed in Figure 3.3.

Hodo et al. [24] provide the full code for the EdgeCodec implementation in a GitHub repository [47]. Therefore the EdgeCodec implementation for this thesis is largely based on this GitHub repository. An important note is that the original EdgeCodec implementation has an optional adversarial training component. For the purposes of this thesis, this is omitted, as preliminary results showed that the adversarial training component did not improve the results or was too difficult to accurately tune in the scope of this thesis.

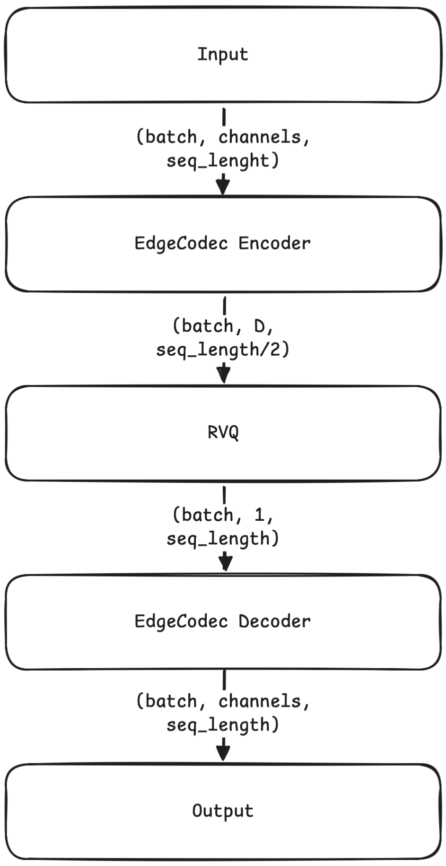


Figure 3.3: EdgeCodec architecture

The EdgeCodec encoder is intentionally lightweight. In essence, the EdgeCodec encoder architecture is fairly similar to the TCN-RNN encoder architecture. Like with the TCN-RNN encoder, the main components of the EdgeCodec encoder are residual units, which contain convolutional layers to capture local patterns. There are, however, notable differences. For one, TCN-RNN has five residual units with increasing dilation compared to the four residual units with steady dilations in the EdgeCodec encoder. Further, the EdgeCodec encoder does not use any weight normalization to optimize for computational efficiency. Another major difference between the EdgeCodec encoder and the TCN-RNN counterpart is the focus on maximizing the compression ability of the encoder. The TCN-RNN encoder employs no downsampling, neither temporal nor channel-wise. The EdgeCodec encoder actually does both. Through increasing strides, the EdgeCodec encoder is able to compress along the temporal axis, and through decreasing the input dimensionality, the EdgeCodec encoder compresses channel-wise. The CR of the EdgeCodec encoder is configurable through these two mechanisms. The detailed EdgeCodec encoder architecture with example configurations is shown in Figure 3.4.

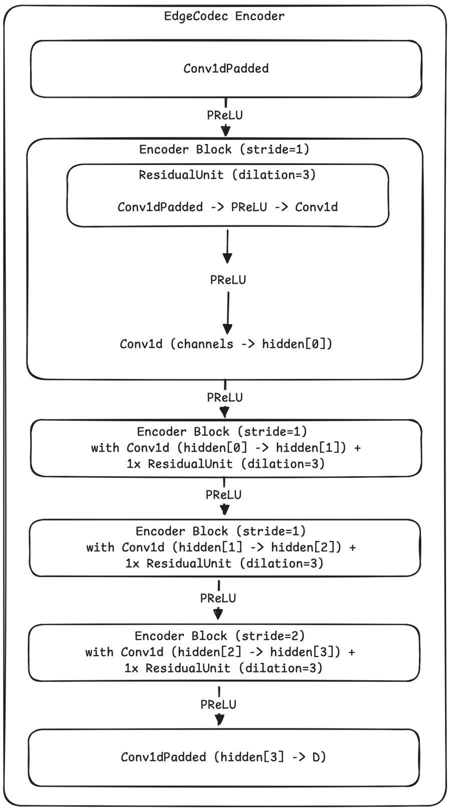


Figure 3.4: EdgeCodec encoder

Because the EdgeCodec encoder is purposefully kept lightweight, the authors compensate through a more complex and heavyweight decoder. The EdgeCodec decoder mainly implements this through channel-wise linear layers that process each channel independently. Two of these channel-wise linear layers are implemented in the EdgeCodec decoder architecture, one at the beginning and one in the middle after the first two decoder blocks. The decoder blocks are also more heavyweight compared to the encoder blocks used in the EdgeCodec encoder. Instead of only one residual unit, the decoder block implements three residual units with increasing dilation to enhance its capabilities in modeling long-range temporal patterns. The downsampling pattern along the temporal and channel-wise axis implemented in the EdgeCodec encoder is mirrored by the EdgeCodec decoder to restore the original input shape. A detailed architectural overview of the EdgeCodec decoder is presented in Figure 3.5.

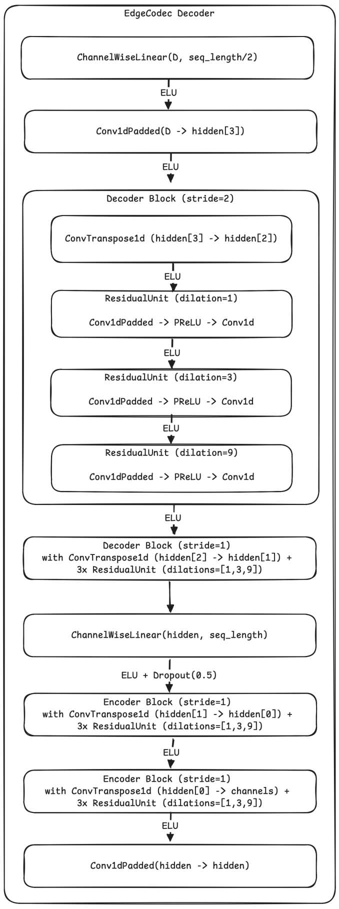


Figure 3.5: EdgeCodec decoder

Although differences in the encoder and decoder structure between the TCN-RNN and the EdgeCodec architecture are substantial, the most substantial difference between the TCN-RNN model and the EdgeCodec model is the use of a quantization mechanism to produce discrete latents. EdgeCodec utilizes a 4-level hierarchical RVQ setup to learn the codebooks. The implementation of RVQ is standardized in most modern neural compression methods through the *vector-quantize-pytorch* library by Phil Wang [48]. Hodo et al. [24] initialize the RVQ method using embedding dimensions equal to the sequence length divided by the temporal compression of the encoder. For example, if the input has a sequence length of 128, and the encoder compresses this with a temporal CR of 2:1, the embedding dimensions for the RVQ method are 64. This enables the RVQ mechanism to assign one codebook per sequence for each channel. The non-differentiability and the resulting challenging end-to-end training of quantization methods has already been discussed above. RVQ handles this by employing exponential moving average (EMA) training, which enables more stable codebook updates compared to gradient-based training.

The loss function used to train EdgeCodec combines reconstruction error, quantization loss, and optional adversarial loss:

$$L(x, \hat{x}, z, \tilde{z}, y, g) = \alpha \cdot L_{\mathrm{MSE}}(x, \hat{x}) + (1-\alpha) \cdot L_{l1-\mathrm{smooth}}(x, \hat{x}) + \eta \cdot L_{\mathrm{RVQ}}(z, \tilde{z}) + \gamma \cdot L_{\mathrm{adv}}(y, g) \tag{3.2}$$

where x is the input, $\hat{x}$ is the reconstructed output, z and $\tilde{z}$ are the original and quantized latents, y and g represent the discriminator and generator outputs, and α, η, γ are weighting coefficients. As described before, for the purposes of this thesis, the adversarial training loss is omitted.

Because the presented implementation of the EdgeCodec baseline is heavily based on the provided code by the authors and largely reproduced one to one, an experiment to validate the correctness of this reproduction of the EdgeCodec model is omitted.

4
Methods

This chapter describes the methodology used in this thesis. It outlines the proposed implementations and improvement strategies for the baseline neural compression methods, the datasets and downstream use cases used for evaluation and the compression metrics. Furthermore, it describes the experimental setup and training details for all experiments conducted.

4.1 Model Design

It is expected that the EdgeCodec baseline implementation performs much better than the TCN-RNN baseline. This is because of two reasons. First, discrete-latent neural compression methods have been shown to consistently outperform continuous-latent neural compression methods like TCN-RNN. Second, EdgeCodec is optimized for the specific use case of compressing sensor data on the edge, which nicely fits the use case of this thesis. Given this, the proposed improvements upon the baseline models are based on the EdgeCodec architecture.

While it is expected that the EdgeCodec baseline will outperform the TCN-RNN baseline, there are a few noteworthy shortcomings of the EdgeCodec architecture. When looking at Figure 4.2 and Figure 4.3, it becomes apparent that the signals in these two datasets have completely contrasting characteristics. This is not unusual in IoT or automotive data, which is why these types of data are usually referred to as multimodal. The EdgeCodec architecture handles each signal the same. All signals share one encoder, one codebook, and one decoder. It is therefore likely that signal-specific nuances will be hard to capture for the EdgeCodec model, especially for signal types that the model is not primarily designed to handle (for example, discontinuous signals). Another potential limitation of the EdgeCodec architecture for this thesis's use case is that the main optimization goal is reconstruction of the data, rather than downstream task utility. With the UniEdgeCodec architecture, this thesis will primarily address the first of these shortcomings, while the TAEdgeCodec tries to intentionally focus on improving downstream task utility retention.

Both UniEdgeCodec and TAEdgeCodec keep the architecture, including the encoder setup, the decoder setup, and the RVQ setup (aside from the partitioning of the codebook for UniEdgeCodec), the same compared to the EdgeCodec architecture. This is done to ensure comparability and keep the efficient architecture of the EdgeCodec model intact. The specific dimensions and parameters used for the individual experiments are introduced in the next sections, where the experiment setup will be presented.

4.1.1 UniEdgeCodec

At the core of the UniEdgeCodec idea is the desire to group IoT/automotive signals and handle each group with a more specialized codebook and loss function to account for group-specific characteristics more clearly. The idea to partition the codebook for different types of inputs is presented by Jiang2025 et al. [49] in their UniCodec architecture. Here, the authors try to address the problem of handling multi-domain audio data with a single codebook. Building on this idea, UniEdgeCodec implements a partitioning of the codebook by signal type. In practice, this means that each signal type will be assigned a specified range of codes in the codebook. In addition to the partitioning of the codebook, each signal type will receive a specific loss function in the UniEdgeCodec architecture. The signal types that are defined by the UniEdgeCodec architecture are continuous vs. discontinuous signals and smooth vs. irregular signals. Within the scope of this thesis, the classification of these signal types is done manually and not learned.

When looking at the signals presented in Figure 4.2 and Figure 4.3, two key characteristics, where the signals differ from one another, can be identified. One is whether a signal is continuous or discontinuous. Continuous signals like the Temp 1 signal of the Volvo EMOB dataset, or the Press_mm_hg signal of the appliances energy dataset should be easily captured by the MSE + SmoothL1 loss already implemented in the EdgeCodec architecture. Discontinuous signals like the Amps signal of the Volvo EMOB dataset or the Windspeed signal of the appliances energy dataset are likely to cause problems. The loss function of the EdgeCodec architecture heavily favors continuous signals and discourages flat regions and sharp edges. Because of this, an additional loss term is introduced for the discontinuous signals. Here, inspiration is drawn from total variation (TV) regularization. TV regularization, as introduced by Rudin et al. [50], is a popular image processing technique that is applied to preserve edges while removing noise. The assumption of TV regularization is that images are piecewise constant with sparse gradients. Optimizing image processing with TV regularization tries to minimize the total variation to preserve the piecewise constant structure of the image. This idea has seen widespread applications in the image processing domain. One example is the work presented by Liu et al. [51], who use TV regularization in combination with CNNs for image restoration. Building upon the idea of TV regularization, UniEdgeCodec applies the following reconstruction loss function to discontinuous signals:

$$L_{recon}(x, \hat{x}, \lambda_{TV}) = L_{\mathrm{MSE}}(x, \hat{x}) + \lambda \cdot L_{\mathrm{TV}}(\hat{x}) \tag{4.1}$$

With $L_{\mathrm{TV}}(\tilde{x})$ being the mean of the first order differences of the predictions.

The second characteristic to differentiate the signals presented in Figure 4.2 and Figure 4.3 is smoothness. Signals like Temp 1 in the Volvo EMOB dataset, and lights or Press_mm_hg in the appliances energy dataset are classified as smooth.

These signals largely have smooth transitions and no rapid increases or decreases. Signals Rh_out or Windspeed in the appliances energy dataset, on the other hand, are classified as irregular. The transitions of these signals are rapid, leading to spiky and irregular signal behavior. When again looking at the MSE + SmoothL1 loss implemented by the EdgeCodec architecture, it is expected that these rapid changes in signal behavior are difficult to capture. It is expected that the loss implemented by EdgeCodec smooths these signals too much, leading to poorer reconstruction. To combat this, UniEdgeCodec again introduces a specific loss for irregular signals. Here, inspiration from perceptual loss methods is drawn. A perceptual loss is defined as a loss function that estimates some part of human perception with the goal of having the model predict closer to this human perception [52]. This idea is implemented in the UniEdgeCodec architecture by first identifying peaks and valleys through per-batch percentile computation, and second applying a specific MSE loss on these sections. The resulting reconstruction loss function, that is applied to irregular signals in the UniEdgeCodec architecture, is:

$$L_{recon}(x, \hat{x}, \lambda_{peak}) = L_{\text{MSE}}(x, \hat{x}) + \lambda \cdot L_{\text{peak}}(x, \hat{x}) \tag{4.2}$$

Through this grouping of the signals, four different signal types can be defined: smooth & continuous signals, smooth & discontinuous signals, irregular & continuous signals, and irregular & discontinuous signals. For smooth & continuous signals, the original MSE + SmoothL1 loss from EdgeCodec is used. For smooth & discontinuous signals, the presented TV regularization loss is used. For irregular & continuous signals, the presented perceptual loss is used. And lastly, for the irregular & discontinuous signals, a combination of the TV regularization loss and the perceptual loss is used. UniEdgeCodec combines each signal-specific loss and computes a weighted average as a combined reconstruction loss. This means that signal types need to be defined before training, so that each signal can be routed to the specific codebook partition and loss function. The complete reconstruction loss function is computed as follows:

$$\begin{aligned} L_{recon}(x, \hat{x}, \alpha, \lambda_{TV}, \lambda_{peak}, \lambda_{TV+peak}) = \; & \alpha \cdot L_{\text{MSE}}(x, \hat{x}) + (1-\alpha) \cdot L_{l1-\text{smooth}}(x, \hat{x}) \\ & + \lambda_{TV} \cdot L_{\text{TV}}(\hat{x}) \\ & + \lambda_{peak} \cdot L_{\text{peak}}(x, \hat{x}) \\ & + \lambda_{TV+peak} \cdot L_{\text{TV+peak}}(x, \hat{x}) \end{aligned} \tag{4.3}$$

Lastly, the RVQ loss is added, just as in the EdgeCodec architecture. The total loss function for UniEdgeCodec is therefore:

$$L(x, \hat{x}, z, \tilde{z}, \lambda_{TV}, \lambda_{peak}, \lambda_{TV+peak}, \eta) = L_{recon}(x, \hat{x}, \lambda_{TV}, \lambda_{peak}, \lambda_{TV+peak}) + \eta \cdot L_{\text{RVQ}}(z, \tilde{z}) \tag{4.4}$$

4.1.2 TAEdgeCodec

Task-aware or task-oriented neural compression has seen an increase in popularity as more and more data is purely used for downstream task models and not human perception. As Yang et al. [18] note, here perceptual quality is irrelevant and the main focus lies on optimizing for the IB problem introduced in Chapter 3. Making a compression model task-aware is achieved by either pre-training or end-to-end training a task head and adding the task loss to the autoencoder loss [18], [53].

To add task-awareness to the EdgeCodec architecture, an end-to-end approach is implemented, which, depending on the dataset, learns a simple classification or regression head and adds the task-loss to the autoencoder loss. The resulting loss function looks as follows:

$$L(x, \hat{x}, z, \tilde{z}, y, \hat{y}) = \alpha \cdot L_{\text{MSE}}(x, \hat{x}) + (1-\alpha) \cdot L_{l1-\text{smooth}}(x, \hat{x}) + \eta \cdot L_{\text{RVQ}}(z, \tilde{z}) + \gamma \cdot L_{\text{task}}(y, \hat{y}) \tag{4.5}$$

The task loss L_{task} is defined as a smooth l1 huber loss for the regression task and a cross entropy loss for the classification task. The targets are directly loaded from the datasets, so that the reconstructed time series can be fed directly to the task head together with targets to produce task predictions and calculate the specific loss. For the regression task, the complete sequence for each window is constructed so that each timestep values is predicted by the task. The classification task tries to predict anomalies on a window-level. The task head for both tasks is identical and purposefully kept simple to isolate the compression method effect. The complete task head architecture is presented in Figure 4.1.

4.2 Data & Use Cases

This section describes the datasets and downstream use cases used for evaluation in this thesis. The datasets are selected to be relevant to the automotive and IoT domain, while also being publicly available. The downstream use cases are selected to be simple enough to allow for clear comparison of the compression methods, while still being relevant to real industry problems.

4.2.1 Datasets

The goal is to use industry-relevant data while also ensuring that the methods are tested on publicly available data. For this reason, two datasets are used: the Volvo electric mobility (EMOB) dataset and the appliances energy dataset used by Zheng and Zhang [22], [54]. Both datasets are accessed through .csv files. The Volvo EMOB dataset captures battery behaviour within an embedded truck system. Three key signals are captured: “Temp 1” for battery temperature and “Amps” and “Volts” for battery current and voltage. Figure 4.2 shows an excerpt of the test dataset, demonstrating the characteristics of each signal used in the Volvo EMOB dataset. The appliances energy dataset measures appliance energy use in a low-energy building

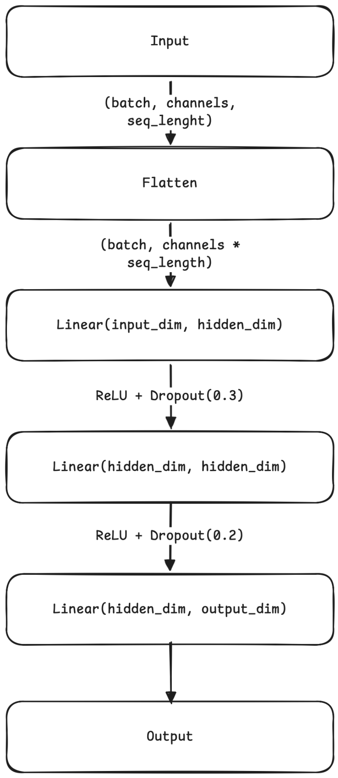


Figure 4.1: Task head architecture

over 4.5 months. Key signals used in this thesis are "Appliances," which measures energy use in Wh and is the primary regression target of the dataset; "lights," which measures the energy use of light fixtures in the house in Wh; "Press_mm_hg," which measures outside pressure in mm Hg; "T_out," which measures outside temperature in Celsius; "RH_out," which measures outside humidity in percent; "Windspeed," which measures outside wind speed in m/s; "Visibility," which measures outside visibility in km; and "Tdewpoint," which measures outside dewpoint temperature. Figure 4.3 presents a snippet of the test split of the appliances energy dataset, showing the behavior of each used signal.

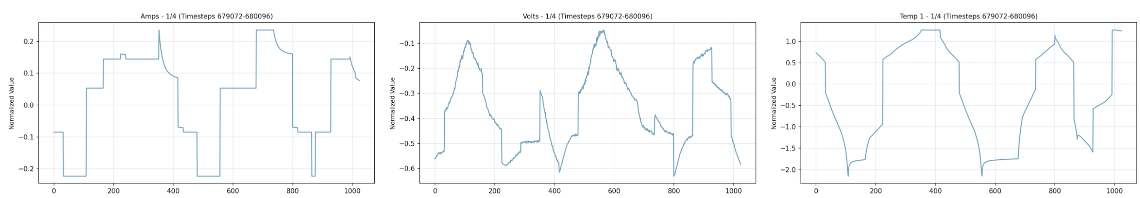
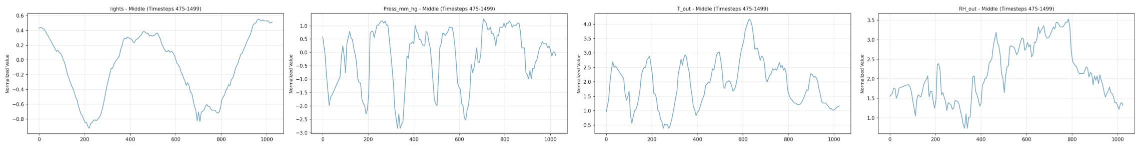
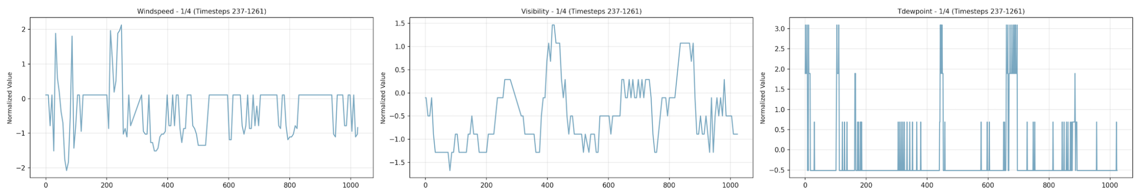


Figure 4.2: Visualization of the Volvo EMOB dataset

For input handling and preprocessing, it is important that the time-series data is segmented into fixed-length windows. These windows are treated as inputs to the model. Overlapping windows can be used to increase the effective training data size and improve reconstruction quality at window boundaries. The appliances energy dataset uses a window size of 144 with stride 1, and the Volvo EMOB dataset uses a window size of 128 with stride 64. Additionally, each channel is normalized independently using statistics computed on the training set. The train/val/test split for the Volvo EMOB and appliances energy datasets is 0.4/0.1/0.5 and 0.81/0.09/0.1, respectively. The values for window size and split ratio for the appliances energy dataset are taken directly from Zheng and Zhang [22]. For the Volvo EMOB dataset, these values result from experiments with different configurations and selection of suitable parameters. The split ratio for the Volvo EMOB dataset heavily emphasizes the test set because, as outlined in the section on downstream task performance, only the test set is used to train downstream models. As the downstream use case for the Volvo EMOB dataset is non-trivial, a larger amount of data is needed for training.



(a) First four signals of the appliances energy dataset



(b) Last three signals of the appliances energy dataset

Figure 4.3: Visualization of the appliances energy dataset

With the setup outlined in the previous paragraph, the two datasets have the following characteristics. The Volvo EMOB dataset is divided across a total of 50 files with 6189262 rows across these files. With a window size of 128 and a stride of 64, a total of 84911 windows are produced. Of these windows, 76993 are normal windows and 7918 are anomaly windows. The train set includes 33964 total windows, 30797 normal and 3167 anomaly windows. The validation set includes 8490 windows, 7699 normal and 791 anomaly windows. Lastly, the test set includes 42457 windows, 38497 normal and 3960 anomaly windows. There are no NULL values in the Volvo EMOB dataset. The appliances energy dataset has a total of 19735 rows. Because of the aggressive stride of 1, this results in 19592 total windows. Of these, 15985 are assigned to the train split, 1776 are assigned to the validation split, and 1974 are assigned to the test split. The appliances energy dataset also has no NULL values.

Each dataset is implemented using a PyTorch “Dataset” and a PyTorch Lightning “LightningDataModule”. Between these two components, the PyTorch “Dataset” is responsible for data preparation steps such as connecting to the data source, loading the data, filtering the data, computing global statistics, and creating windows. The PyTorch Lightning “LightningDataModule” encapsulates this logic and splits the data to produce machine-learning-ready train/validation/test datasets.

4.2.2 Downstream Use Cases

Comparing the compression methods based on downstream task performance requires concrete downstream use cases. The use cases and the models that implement them must be simple enough to enable clear evaluation of the compression methods, while still being relevant to real industry problems. For this reason, two downstream use cases are defined.

For the appliances energy dataset, the dataset-defined regression task of predicting the “Appliances” signal is implemented. All aforementioned signals are used as features, and reconstructed windows (decoder outputs) are fed to a random forest regressor. The random forest provides a balance between simplicity for comparison and adequate predictive power. The regressor is implemented with `scikit-learn` and initialized with `n_estimators=100` and `max_depth=10`. The regressor is evaluated using standard metrics such as MSE and R2 but the final utility retention comparison is based on the R2 metric because of its robustness and scale invariance.

The Volvo EMOB dataset is used for an anomaly-detection classification task. Labels are produced using a Volvo internal interquartile-range outlier detection script. This script uses the 3-sigma rule as a threshold to capture roughly 99.7% of normal data and classify the rest as anomalies. All three signals are used as features to train a random forest classifier, implemented identically to the regressor (same hyperparameters: `n_estimators=100`, `max_depth=10`). Because there is class imbalance in the Volvo EMOB dataset, the `class_weight='balanced'` parameter is additionally used. The classifier is evaluated using standard metrics such as accuracy, precision, recall, and F1-score. The final utility retention calculations for this use case will be based on the F1-score, as it provides a good balance between precision and recall, which gives a comprehensive picture of the model’s performance on the imbalanced dataset.

4.3 Experiments

To answer the research questions outlined in Chapter 1, this thesis follows a three-phase experimental strategy. Following the taxonomy introduced by Stol and Fitzgerald [55], this thesis uses a controlled laboratory experiment to compare neural compression architectures under a fixed benchmark setup. The emphasis is on isolating the effect of architectural choices on the defined metrics.

First, two baseline neural compression methods are selected to represent the main design directions discussed in the related work, namely a continuous-latent method and a discrete-latent method. Zheng and Zhang [22] are used as the representative continuous-latent baseline, while EdgeCodec by Hodo et al. [24] is used as the representative discrete-latent baseline because of its emphasis on efficient edge-oriented compression.

In the second phase, both baseline methods are reproduced according to the specifications of the respective authors and evaluated on IoT and automotive time-series data. Their compressed representations are then used to train downstream machine learning models, which are also trained on the corresponding uncompressed data.

Comparing the downstream performance of these models provides a direct measure of how much utility is retained after compression and reconstruction.

In the third phase, the two baseline methods are examined in greater detail to identify their main failure modes with respect to reconstruction quality and downstream utility retention. Based on these findings, targeted modifications to the quantization strategy and loss function are implemented in two new exploratory variants, named *UniEdgeCodec* and *TAEdgeCodec*. These variants are then evaluated using the same experimental setup as the original baselines in order to assess whether the proposed changes improve the observed limitations.

4.3.1 Comparison and Metrics

There are two dimensions along which the compression methods are compared. First, the *compression performance* of both methods is assessed. The CR and reconstruction error are primarily used to evaluate compression performance. These metrics align with the evaluation standards in the neural compression literature. Additioanlly, average correlation between the reconstructed signals and perceptual reconstruction quality are taken into account, to give a broader picture of the qualities of the reconstructed artifacts. All metrics are measured by training the models on a reconstruction task and evaluating the reconstruction quality and effective CR. For reconstruction error, mean squared error (MSE) is used. Within this thesis, the mean-reduced variant of MSE is used. The MSE loss is defined as:

$$L_{\mathrm{MSE}} = \frac{1}{T \cdot D} \sum_{t=1}^{T} \sum_{d=1}^{D} \left(x_t^d - \hat{x}_t^d \right)^2 \tag{4.6}$$

Besides reconstruction error, CR is the other important metric to evaluate compression performance. Within this thesis, CR is defined exactly as described by the authors of the EdgeCodec paper [24]. CR is defined as:

$$\#\text{bits} = \lceil \log_2(k) \rceil, \quad \mathrm{CR} = \frac{A \cdot B}{\#\text{bits} \cdot N} \tag{4.7}$$

Because this equation is not immediately intuitive, a numerical example is provided. The architecture presented by Hodo et al. [24] compresses the input using both the encoder and the quantization module. In the EdgeCodec paper, the input has 36 channels and a window size of 800 time steps. The output of the encoder is a downsampled version of this input with dimensions 9 channels $\times$ 100 time steps. This means that the encoder alone compresses with a CR of $(800/100) \cdot (36/9) = 32\times$. In the quantization setup, the authors compress with precision A, which corresponds to 32-bit precision of the input signals. The number of bits per code depends on the codebook size k. For a codebook of size 768, the bits per code are roughly 10. The total number of transmitted codes, N, is calculated by multiplying the number of quantizers by the number of input channels to the quantizers. Because the encoder outputs 9 channels and the architecture presented by Hodo et al. [24] uses four quantizers, N is equal to 36. Applying the CR equation gives $\frac{9 \cdot 100 \cdot 32}{36 \cdot 10} = 80\times$.

The total CR is then obtained by multiplying the encoder CR by the quantizer CR, resulting in a total CR of $32 \cdot 80 = 2560\times$, as presented in the paper.

Second, the *downstream task performance* when using compressed representations of the training data for selected downstream models from both methods is evaluated. This includes metrics such as accuracy for classification tasks or MSE for regression tasks. To measure this, the downstream models are trained on uncompressed data and compressed data using different neural compression methods. By comparing the utility loss across these approaches, the utility retention of the various neural compression methods can be measured.

An important note regarding comparability of the neural compression methods, especially for downstream task performance, is that the methods produce different types of compressed representations. The TCN-RNN baseline produces a continuous latent representation, while the EdgeCodec implementation produces discrete latents. To ensure a fair comparison, the compressed data is reconstructed using the learned decoders before being used as training data for the downstream models. This setup makes it possible to measure which approach retains more downstream task utility after compression and reconstruction.

Another important note for the evaluation of downstream task performance is that only the test portion of the available datasets is used when training downstream models. The train and validation splits are used during training of the neural compression methods. The test set is then used to evaluate the neural compression method. Because the neural compression methods have seen the train and validation splits during training, these splits cannot be used when deploying the neural compression methods to compress data for downstream tasks. For this reason, only the test split is used as input, either compressed or uncompressed, to the downstream models.

4.3.2 Training Parameters

In order to run the training there are certain hyperparameters which need to be decided upon for all variations of the experiments. These are parameters which will be fixed at the same value for all experiments. These parameters include the batch size and the random seed. The batch size used is always 64 and the random seed is 42, 100 and 200 respectively. The final results will be the average of the results from each seed. Gradient clipping is also applied with a max norm of 1.0, and mixed precision training is enabled with a precision of 32 bits for all experiments. The optimizer used for all experiments is Adam and the learning rate is set to 1e-3 for all experiments on the Emob dataset and the TCN-RNN model on the smart home dataset. All other experiments on the smart home dataset use a learning rate of 5e-4. This is done to account for the fact that the smart home dataset is smaller and more complex, which makes it more difficult to train on. For all models which need to balance the MSE loss with other losses, the alpha parameter is set to 0.5, meaning that the MSE loss and the other loss are weighted equally. For all models which need to balance the reconstruction loss with the RVQ loss, the eta parameter is set to 0.25, meaning that the reconstruction loss is weighted more heavily than the RVQ loss. The specific hyperparameters for each model are introduced in more detail in the next sections.

TCN-RNN For the baseline model, the relevant hyperparameters were set based on the implementation in the original paper by Zheng and Zhang [22]. The TCN encoder is implemented with 5 residual blocks, with a channel progression of [3, 3, 3, 3, 3] for the Emob dataset and [7, 7, 7, 7, 7] for the smart home dataset. The kernel size is set to 4 and the dropout is set to 0.1. The linear bottleneck compresses the channel dimension from the orignal number of channels to 1, resulting in a CR of x:1 where x is the initial number of channels. The GRU decoder is implemented with a hidden size of 3 and 7 respectively across 1 layer. The loss function used for training is the sum-reduced MSE loss, as described in the methods section. The decoder is not bidirectional, and the sequence is reversed before being fed to the decoder. The maximum number of epochs is set to 500, with an early stopping mechanism that stops training if the validation loss does not improve for 10 consecutive epochs with a minimum improvement threshold of 1e-4 and the l2 regularization strength is set to 0. The learning rate scheduler used is cosine annealing with 500 epochs and a minimum learning rate of 1e-6. The optimizer uses betas of (0.9, 0.999) and epsilon of 1e-08. The pytorch lightning trainer is configured with the same maximum epochs and early stopping mechanism as described before.

EdgeCodec For the EdgeCodec baseline, the hyperparameters were initially set based on the original implementation by Hodo et al. [24]. However, the paper is not fully clear on the exact hyperparameters used for training and the publicly available codebase does not provide exact details about the training. Therefore, while the original implementation served as a starting point, the hyperparameters were adjusted where it was considered necessary or beneficial to fit the data. The codebook size and betas are the two parameters which are directly taken from the original implementation, all other parameters have been adjusted.

The channel progression in the encoder is [7, 7, 7, 7] for the smart home dataset and [3, 3, 3, 3] for the Emob dataset, matching the number of initial channels. The quantizer has 2 stages of residual quantization with a codebook size of 768 per quantizer and an embedding dimension of 32 for the Emob dataset and 72 for the smart home dataset. This is done in order to align the embedding dimension with the sequence length divided by the temporal CR of the encoder, as described earlier in Section 3. The decoder then uses a identical channel progression as the encoder. The EdgeCodec uses an optimizer with betas of (0.9, 0.98) and epsilon of 1e-8. The pytorch lightning trainer is configured with maximum epochs of 80. The early stopping mechanism is configured with a patience of 20 epochs and a minimum delta of 1e-2. The scheduler used is a cosine annealing scheduler with a maximum of 500 and a minimum learning rate of 1e-6.

UniEdgeCodec The UniEdgeCodec remains largely similar to the EdgeCodec in terms of architecture and hyperparameter settings. The values used for the encoder, decoder, optimizer and the pytorch lightning trainer are the same as those used for the EdgeCodec. The quantizer uses a codebook with the size of 1000 partitioned into smooth-continuous [0 - 300], smooth-discontinuous [300 - 300], irregular-continuous [300 - 600] and irregular-discontinuous [600 - 1000] partitions. The signals are categorized as follows: lights and HR_out are categorized as smooth-continuous, Press_mg_hg and T_out are categorized as irregular-continuous and Windspeed, Visibility and Tdewpoint are categorized as irregular-discontinuous. The embedding dimensions and commitment cost of the RVQ varies between datasets as the Emob dataset has a embedding dimension of 64 and a commitment cost of 0.75 while the smart home dataset has an embedding dimension of 72, similar to the EdgeCodec implementation, and a commitment cost of 0.35. The scheduler used is a cosine annealing scheduler with a maximum of 80 and a minimum learning rate of 1e-6. Additionally, the Uni-EdgeCodec uses a lambda of 0.1 for the tv loss, a peak weight of 1.0 for the peak loss, a peak percentile of 90, a discontinuous weight of 10.0 for the TV loss, a perceptual weight of 1.0 for the peak loss and a discontinuous perceptual weight of 1.0 for the combined TV and peak loss.

TAEdgeCodec The task aware EdgeCodec also share many hyperparameters with the base EdgeCodec model. These include the parameters for the encoder, decoder, optimizer, scheduler and the pytorch lightning trainer. The quantizer also has a codebook with the size of 768 and an embedding dimension of 64. The main difference is the training module which now has a beta for balancing the task loss. This value is set to 1e-2 and the dimension for the task head is set to 128.

4.3.3 Compression Ratio

Certain parameters have been set with the intent of roughly alligning the CR of each codec so that their results will be comparable. For this reason, the codebook size and the number of quantizers have been modified. The CR can be computed as follows:

$$\text{original\_size} = \text{channels} \times \text{timesteps} \times 32 \tag{4.8}$$

$$\text{compressed\_size} = \lceil \log_2(\text{codebook\_size}) \rceil \times \text{channels} \times \text{num\_quantizers} \tag{4.9}$$

$$\text{CR} = \frac{\text{original\_size}}{\text{compressed\_size}} \tag{4.10}$$

Table 4.1 presents the computed CR for the three architectures evaluated on the Volvo EMOB dataset, demonstrating that the architectures achieve comparable compression performance despite their different internal configurations.

Table 4.1: Compression Ratio Comparison for EMOB Dataset (3 channels, 128 timesteps)

Model	**Codebook Size**	**Emb. Dim.**	**Num Quant.**	**CR**
EdgeCodec	768	32	2	213.7×
UniEdgeCodec	1000	64	2	204.8×
TAEdgeCodec	768	64	2	213.7×

Similarly, Table 4.2 shows the computed CR for the appliances energy dataset. Despite the larger number of channels and longer window size compared to the EMOB dataset, all three architectures maintain comparable compression performance, achieving between 240× and 230× compression.

Table 4.2: Compression Ratio Comparison for Appliances Energy Dataset (7 channels, 144 timesteps)

Model	Codebook Size	Emb. Dim.	Num Quant.	CR
EdgeCodec	768	72	2	240.4×
UniEdgeCodec	1000	72	2	230.4×
TAEdgeCodec	768	64	2	240.4×

4.4 Threats to Validity

The validity of empirical software engineering research is commonly divided into internal validity, construct validity, conclusion validity, and external validity. Internal validity concerns whether the observed effect is truly caused by the treatment or whether it can be explained by other uncontrolled factors. In controlled laboratory experiments, such as the experiments conducted in this thesis, this is a central concern. Although confounding factors can still occur, the experimental setup is designed to minimize them and isolate the causal effect.

External validity usually suffers when the focus is on internal validity. External validity concerns the generalizability of the findings. While different datasets and use cases are used to provide some variation, the focus of this thesis is to observe the effects of architectural choices on selected metrics. The external validity is therefore limited and should be examined more carefully in future work.

Conclusion validity, which concerns the soundness of the empirical findings, is an important issue in experiments that depend on many hyperparameters and training parameters. To avoid fishing for satisfactory results or p-hacking, the experimental setup is fixed as much as possible before the experiments are run. This helps to preserve conclusion validity, but the results may still differ when the setup is changed.

Lastly, construct validity, or conceptual correctness, concerns whether the chosen metrics actually measure the intended concepts. In this thesis, compression quality is operationalized through reconstruction error and CR, while utility retention is approximated through downstream task performance after reconstruction. These measures are standard in the neural compression literature, but they only capture the intended constructs indirectly. For example, low reconstruction error does not necessarily imply semantic preservation, and downstream performance depends not only on the compressed representation but also on the decoder and the selected downstream tasks.

5

Results

Within this chapter, the results of the experiments are presented. They are ordered by importance, starting with the key findings and then comparing each neural compression method per metric.

5.1 Key Findings

UniEdgeCodec shows the strongest and most consistent reconstruction correlation of the tested methods. For EMOB, across the three seeds, its mean correlation is 0.960 ± 0.007, compared with 0.780 ± 0.034 for EdgeCodec and 0.765 ± 0.035 for TAEdgeCodec. This is especially notable for irregular and discontinuous signals like "Amps" in the Volvo EMOB dataset or "Visibility" in the appliances energy dataset. At the same time, the reconstruction MSE of smooth and continuous signals tends to suffer somewhat from the improved handling of irregular and discontinuous signals. The improvement in correlation also leads to better perceptual reconstruction quality, which becomes obvious when comparing irregular and discontinuous signals with smooth and continuous signals. Overall, the UniEdgeCodec model is more stable in terms of metric variance and training convergence. Across the board, mean correlation, utility retention, and training convergence are more stable for the UniEdgeCodec model, and mean MSE is more stable in one of the two examined cases.

TAEdgeCodec does not consistently improve upon the EdgeCodec model in terms of utility retention and reconstruction performance. Additionally, its behaviour is strongly seed dependent. Across the three EMOB seeds, its mean correlation is 0.765 ± 0.035 and its utility retention is $67.0 \pm 13.8\%$. The results for the appliances energy dataset tell a similar story. Here the mean correlation is 0.515 ± 0.070, while utility retention remains high at $89.4 \pm 0.5\%$. Also notable is the poor convergence of the TAEdgeCodec model for seed 100 on the Volvo EMOB dataset. The model therefore seems capable of preserving downstream utility, but the balance between reconstruction and task performance is unstable and depends on both the dataset and the seed.

EdgeCodec remains a strong reconstructor, especially on the regression task. On EMOB, its mean correlation is 0.780 ± 0.034 and its utility retention is $65.1 \pm 10.8\%$. For the appliances energy dataset, the corresponding values are 0.635 ± 0.061 and $90.1 \pm 1.9\%$. The model is therefore competitive on reconstruction, but its utility retention is less stable than UniEdgeCodec on EMOB, and its reconstruction is generally weaker on the more difficult channels. The training stability of EdgeCodec lies somewhere between the stable UniEdgeCodec model and the unstable TAEdgeCodec model.

TCN-RNN is consistently weaker than the discrete-latent methods under compression. On EMOB, it reaches a mean correlation of 0.726 with a compression ratio of only 3.0x, while the discrete-latent methods operate at roughly 205x to 214x compression. For the appliances energy dataset, the baseline reaches a mean correlation of 0.651 and a compression ratio of 7.0x, again below the discrete-latent methods in both reconstruction quality and compression strength.

Across the EMOB runs, utility retention is more closely tied to reconstruction quality than for the runs on the appliances energy dataset. For the EMOB use case, it generally holds that better reconstruction MSE and better reconstruction correlation lead to better utility retention.

5.2 Empirical Evidence

Below, selected empirical results supporting the key findings are presented. The complete empirical results are provided in Appendix A. The tables summarize the reconstruction metrics and downstream utility retention values that motivate the discussion in the next chapter. First, the findings for the proposed models UniEdgeCodec and TAEdgeCodec are presented; then the two baselines EdgeCodec and TCN-RNN are compared.

5.2.1 UniEdgeCodec and TAEdgeCodec

Table 5.1 summarizes the two proposed models across both datasets and the three used seeds. UniEdgeCodec achieves the strongest and most stable EMOB correlation, while TAEdgeCodec is more variable across seeds and datasets. For the appliances energy dataset, both models preserve a large fraction of the downstream regression utility, but UniEdgeCodec does so with more stable reconstruction behavior, while TAEdgeCodec generally performs better in terms of utility retention.

Table 5.1: Mean reconstruction and utility results for UniEdgeCodec and TAEdgeCodec across seeds. Values are reported as mean ± standard deviation over seeds.

Model	EMOB MSE ↓	EMOB Corr. ↑	EMOB Util.	Appliances Energy Dataset MSE ↓	Appliances Energy Dataset Corr. ↑	Appliances Energy Dataset Util.
UniEdgeCodec	0.030 ± 0.008	0.960 ± 0.007	63.8 ± 0.8%	0.611 ± 0.151	0.664 ± 0.016	83.8 ± 0.1%
TAEdgeCodec	0.047 ± 0.047	0.765 ± 0.035	67.0 ± 13.8%	0.538 ± 0.060	0.515 ± 0.070	89.4 ± 0.5%

The numbers show two different types of behaviour. UniEdgeCodec is the more stable model, especially on EMOB, where the standard deviation in correlation is only 0.007. When training convergence is taken into account as well, this stability is further emphasized. UniEdgeCodec consistently converges over 60–70 epochs, while TAEdgeCodec can collapse and converge at epoch 5, as is the case with seed 100 and the Volvo EMOB dataset. The complete trainig convergence data is shown in Table 5.2 TAEdgeCodec can reach strong utility retention, but the much larger EMOB standard deviation in utility retention makes that result less reliable as a general pattern.

Table 5.2: Epoch convergence (final epoch and validation loss) for UniEdgeCodec and TAEdgeCodec across datasets and seeds.

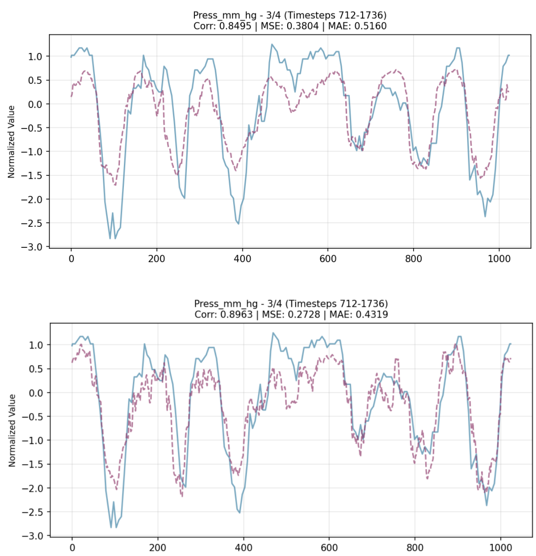
Model	Dataset	Seed	Final epoch	Val. loss
UniEdgeCodec	EMOB	s42	61	0.0713
UniEdgeCodec	EMOB	s100	58	0.0746
UniEdgeCodec	EMOB	s200	59	0.0873
UniEdgeCodec	Appliances energy	s42	71	1.2485
UniEdgeCodec	Appliances energy	s100	58	1.6870
UniEdgeCodec	Appliances energy	s200	62	1.5236
TAEdgeCodec	EMOB	s42	61	0.0264
TAEdgeCodec	EMOB	s100	5	0.0909
TAEdgeCodec	EMOB	s200	50	0.0257
TAEdgeCodec	Appliances energy	s42	11	1.1814
TAEdgeCodec	Appliances energy	s100	40	1.0563
TAEdgeCodec	Appliances energy	s200	31	1.1054

Looking at the seed-specific metrics for each dataset, it can also be seen that there appears to be a correlation between reconstruction quality and downstream task utility retention in the Volvo EMOB use case. Utility retention tends to deteriorate when reconstruction MSE and reconstruction correlation become worse for the Volvo EMOB use case. The same is not true for the appliances energy dataset. Table 5.3 shows these signal-specific metrics.

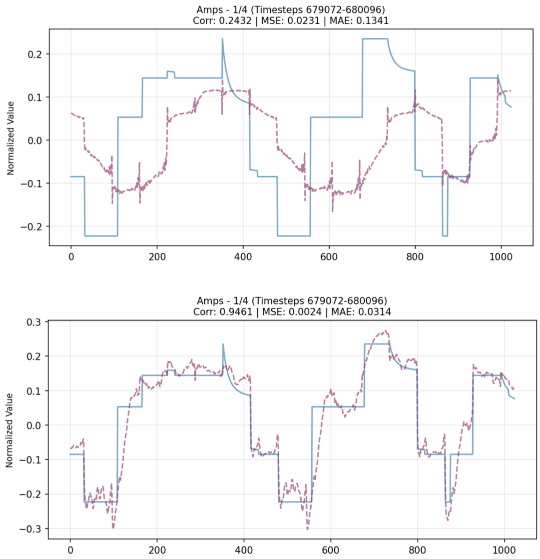
Table 5.3: Per-seed reconstruction and downstream utility for UniEdgeCodec and TAEdgeCodec

Dataset	Model (seed)	Meaned MSE ↓	Meaned Corr. ↑	Utility retention ↑
EMOB	UniEdgeCodec (s42)	0.024068	0.962822	62.85%
EMOB	UniEdgeCodec (s100)	0.026717	0.965547	64.15%
EMOB	UniEdgeCodec (s200)	0.039726	0.953058	64.43%
EMOB	TAEdgeCodec (s42)	0.021217	0.771536	73.55%
EMOB	TAEdgeCodec (s100)	0.101188	0.726986	51.08%
EMOB	TAEdgeCodec (s200)	0.020049	0.795126	76.35%
Appliances energy	UniEdgeCodec (s42)	0.467329	0.677110	83.75%
Appliances energy	UniEdgeCodec (s100)	0.768442	0.646833	83.85%
Appliances energy	UniEdgeCodec (s200)	0.596645	0.668311	83.76%
Appliances energy	TAEdgeCodec (s42)	0.604076	0.436676	89.95%
Appliances energy	TAEdgeCodec (s100)	0.487089	0.569029	89.33%
Appliances energy	TAEdgeCodec (s200)	0.522684	0.539953	88.96%

When looking at signal-specific failure modes, a pattern becomes clear. UniEdgeCodec specifically targets the weakness of the TAEdgeCodec model in reconstructing irregular and discontinuous signals. Signals that are smooth and continuous are generally well reconstructed by the TAEdgeCodec model. Examples of such signals are the “Temp 1” signal in the Volvo EMOB dataset and the “Press_mm_hg” signal in the appliances energy dataset. Figure 5.1a presents the reconstruction of the “Press_mm_hg” signal for both the TAEdgeCodec and UniEdgeCodec models. A slight improvement for UniEdgeCodec is visible, but in general both models seem capable of handling these types of signals.



(a) Reconstruction of the “Press_mm_hg” signal



(b) Reconstruction of the “Amps” signal

Figure 5.1: Reconstruction of representative smooth and irregular signals. In each subfigure, the TAEdgeCodec reconstruction is shown on the left and the UniEdgeCodec reconstruction is shown on the right.

Figure 5.1b shows the reconstruction of the “Amps” signal, which exemplifies the behavior of these models for irregular and discontinuous signals. Here a clear improvement of perceptual reconstruction quality and reconstruction MSE from the TAEdgeCodec model to the UniEdgeCodec model can be seen.

The pattern that the irregular and discontinuous signals (“Amps” for the Volvo EMOB dataset and “Windspeed”, “Visibility”, “Tdewpoint” for the appliances energy dataset) are consistently improved by the UniEdgeCodec model holds, when observing the signal-specific metrics holistically. It should be noted that the smooth and continuous signals tend to pay a bit in terms of reconstruction quality for the improvements in the irregular and discontinuous signals, and that signals like “lights”, “RH_out”, and “T_out” are significantly better when comparing the TAEdgeCodec metrics to the UniEdgeCodec metrics. Table 5.4 presents the mean MSE and correlation, with standard deviations over seeds, for each signal in both the EMOB and appliances energy datasets.

Table 5.4: Per-signal reconstruction metrics for UniEdgeCodec and TAEdgeCodec across EMOB and appliances energy datasets.

Dataset	Signal	UniEdgeCodec MSE ↓	UniEdgeCodec Corr. ↑	TAEdgeCodec MSE ↓	TAEdgeCodec Corr. ↑
EMOB	Amps	0.008 ± 0.001	0.925 ± 0.008	0.046 ± 0.001	0.344 ± 0.036
EMOB	Volts	0.026 ± 0.012	0.987 ± 0.006	0.022 ± 0.022	0.989 ± 0.011
EMOB	Temp 1	0.057 ± 0.023	0.970 ± 0.012	0.074 ± 0.091	0.961 ± 0.048
Appliances Energy	lights	0.209 ± 0.013	0.247 ± 0.131	0.161 ± 0.044	0.473 ± 0.158
Appliances Energy	Press_mm_hg	0.381 ± 0.084	0.872 ± 0.030	0.443 ± 0.012	0.893 ± 0.029
Appliances Energy	T_out	1.139 ± 0.283	0.806 ± 0.044	0.578 ± 0.042	0.877 ± 0.009
Appliances Energy	RH_out	1.528 ± 0.570	0.663 ± 0.101	0.749 ± 0.077	0.798 ± 0.035
Appliances Energy	Windspeed	0.354 ± 0.071	0.706 ± 0.077	0.684 ± 0.108	0.085 ± 0.300
Appliances Energy	Visibility	0.307 ± 0.108	0.730 ± 0.081	0.526 ± 0.215	0.483 ± 0.123
Appliances Energy	Tdewpoint	0.358 ± 0.012	0.625 ± 0.019	0.624 ± 0.067	-0.002 ± 0.033

5.2.2 EdgeCodec

Table 5.5 shows the baseline EdgeCodec results. The model generally performs well on reconstruction tasks, especially reconstruction MSE, and is surprisingly competitive with the TAEdgeCodec and UniEdgeCodec models at utility retention without being specifically trained for it. EdgeCodec also exhibits similar instability to the TAEdgeCodec model, with standard deviations generally being larger than those observed with the UniEdgeCodec model. This is also observable in the training convergence stability, though to a lesser extent than for the TAEdgeCodec model. The EdgeCodec model never completely collapses, but can converge anywhere between epoch 23 and epoch 75 in the appliance energy case and between epoch 24 and epoch 40 in the Volvo EMOB case.

Table 5.5: Mean reconstruction and utility results for EdgeCodec across seeds. Values are reported as mean ± standard deviation over seeds.

Model	EMOB MSE ↓	EMOB Corr. ↑	EMOB Util.	Appliances Energy Dataset MSE ↓	Appliances Energy Dataset Corr. ↑	Appliances Energy Dataset Util.
EdgeCodec	0.038 ± 0.028	0.780 ± 0.034	65.1 ± 10.8%	0.490 ± 0.072	0.635 ± 0.061	90.1 ± 1.9%

The EdgeCodec results are a useful baseline because they show that strong reconstruction does not automatically translate into the best downstream behaviour. On EMOB, the model is competitive but not as stable as UniEdgeCodec. For the appliances energy dataset, the reconstruction is solid and the regression utility retention is high, which makes the model a strong reference point for the UniEdgeCodec and TAEdgeCodec implementations.

The observation made for the TAEdgeCodec and UniEdgeCodec models about a correlation between reconstruction quality and utility retention in the Volvo EMOB case can also be made for the EdgeCodec model. Further, the EdgeCodec model also exhibits similar struggles to the TAEdgeCodec model when reconstructing irregular and discontinuous signals, but performs well on the reconstruction of smooth and continuous signals. The complete per-signal reconstruction metrics are displayed in Table 5.6.

Table 5.6: Per-signal reconstruction metrics for EdgeCodec across EMOB and appliances energy datasets.

Dataset	Signal	EdgeCodec MSE ↓	EdgeCodec Corr. ↑
EMOB	Amps	0.045 ± 0.003	0.376 ± 0.074
EMOB	Volts	0.019 ± 0.018	0.991 ± 0.009
EMOB	Temp 1	0.050 ± 0.066	0.973 ± 0.035
Appliances Energy	lights	0.125 ± 0.042	0.683 ± 0.061
Appliances Energy	Press_mm_hg	0.354 ± 0.041	0.906 ± 0.012
Appliances Energy	T_out	0.630 ± 0.177	0.907 ± 0.026
Appliances Energy	RH_out	0.696 ± 0.189	0.877 ± 0.039
Appliances Energy	Windspeed	0.559 ± 0.120	0.426 ± 0.227
Appliances Energy	Visibility	0.494 ± 0.107	0.545 ± 0.145
Appliances Energy	Tdewpoint	0.573 ± 0.018	0.104 ± 0.182

5.2.3 TCN-RNN

The TCN-RNN baseline is reported separately because it is not evaluated across the same seed set as the discrete-latent models. Its lower compression ratio makes it a weaker comparator for the compression-utility trade-off, but it still provides a clear lower baseline for reconstruction quality.

Table 5.7: TCN-RNN reconstruction and compression results. The baseline is reported from a single run in each dataset, so no seed standard deviation is available.

Dataset	Meaned MSE ↓	Meaned Corr. ↑	Utility	Compression ratio
EMOB	0.123	0.726	n/a	3.0x
Appliances Energy Dataset	0.771	0.651	n/a	7.0x

The TCN-RNN baseline is clearly behind the discrete-latent models in compression efficiency. Even where reconstruction is acceptable, the much lower compression ratio makes it less compelling than the discrete alternatives. Increasing the compression ratio from 3x to only 7x already massively deteriorates the reconstruction quality.

6
Discussion

Examining the results in light of the research questions, RQ1 relates to the performance of the selected neural compression methods on multimodal IoT data specifically. While this question is very broad and difficult to answer in a general sense, the results show that the discrete methods, EdgeCodec, UniEdgeCodec and TAEdgeCodec, are able to achieve high compression ratios of 200x+ while maintaining good reconstruction quality. The results also highlight the fact that multimodal data in particular creates new challanges for compression methods, as the different signals can have very different characteristics, and thus require tailored compression strategies. RQ2 relates to the utility retention of the selected neural compression methods. As can be seen in the results, the utility retention is highly variable across random seeds, and the relationship between reconstruction quality and utility retention is complex and task-dependent. It is hard to definitively say which architecture favours utility retention the most, but it is clear that minimizing the loss of task relevant information is difficult at high CRs. RQ3 relates to the limitations of the existing methods (EdgeCodec) and how to mitigate these limitations. The results here show that the partitioned codebook design of the UniEdgeCodec is somewhat effective at improving signal reconstruction quality, in terms of correlation, training stability and perceptual quality but does not always lead to improved utility retention. Using a task loss term, as in the TAEdgeCodec, can lead to improved utility retention, but struggles with training stability and reconstruction quality generally suffers as a result.

The superior reconstruction correlation achieved by UniEdgeCodec (0.960 ± 0.007 on EMOB) compared with its competitors can be explained by its design objective of optimizing perceptual quality, particularly for irregular and discontinuous signals. Because perceptual quality is closely correlated with reconstruction correlation, as can be seen in the visualizations of the reconstructed signals, explicit optimization for this objective should lead to improved correlation metrics across all signal types. This consistent preservation of temporal structure is especially beneficial for downstream anomaly detection tasks, where classification reliability depends on accurately capturing temporal patterns.

The model's architectural choice of partitioning the codebook, and thus preventing signals from competing within the same codebook, together with the regularizing effect of the loss function, contributes to the training stability observed across all random seeds. These factors likely resulted in the UniEdgeCodec having both the lowest metric variance and the most reliable convergence behavior across the experimental runs.

This being said, there are notable downsides to having a partitioned codebook, as is exemplified in both use case. UniEdgeCodec consistently achieves the lowest utility retention in the appliances energy use case, mainly, because of the poor reconstruction of the outside temperature. When calculating the pearson correlation between the "T_out" signal, and utility retention for the appliances energy use case (r=0.814, p=0.0076), it becomes clear that the outside temperature is massively important for this use case. A similar behavior can be seen in the Volvo EMOB use case. Here the overall reconstruction quality is even better for the UniEdgeCodec compared to the TAEdgeCodec and the EdgeCodec models. But the utility retention is still better, although unreliably so, for the TAEdgeCodec and EdgeCodec models. Again, individual signals seem to be unevenly important for the downstream task. While EdgeCodec in most cases reconstructs "Temp 1" adequately, this comes at the cost of poor reconstruction of channels with sparse, irregular patterns like the "Amps" signal. Looking at the seed where EdgeCodec performs worse in terms of utility retention for the Volvo EMOB use case, it can be seen that the correlation for the "Temp 1" signal degrades to 0.93, compared with a correlation of 0.99 for seed 42, while the correlation for the "Amps" signal improves to 0.34, compared with a correlation of 0.32 for seed 42. This points to a correlation between "Temp 1" and the anomalies to be detected, but it also shows that the EdgeCodec architecture fundamentally has to trade off significantly between reconstructing smooth, continuous signals and irregular, discontinuous signals. For the UniEdgeCodec model, this means that even though it can achieve a higher total correlation and comparable or better reconstruction MSE on many signals, specific signals matter more for certain downstream tasks than others.

The mixed and seed-dependent results of TAEdgeCodec would suggest that multi-objective neural compression struggles when with producing stable result. This reveals a fundamental challenges of models such as this one. The more complex optimization landscape that arises from jointly learning compression and downstream task performance creates inherent trade-offs. The model must allocate limited representational capacity between reconstruction fidelity and task-specific features. This likely explains the observed pattern of worse reconstruction compared with UniEdgeCodec and EdgeCodec, instability in utility retention, and occasional convergence failures, such as the early collapse observed in seed 100 for the Volvo EMOB data. That said, the task-aware nature of TAEdgeCodec does occasionally lead to very good utility retention results. This suggests that, when the optimization objectives can be jointly satisfied, the approach can succeed. However, the substantially higher variance across seeds indicates that the balance between reconstruction and task optimization is fundamentally unstable without the additional architectural constraints provided by UniEdgeCodec's per-signal codebook design. It is fair to say that, for this model, multimodal data compression and task-aware learning exceed the model's representational capacity.

EdgeCodec's strong reconstruction performance in terms of mean MSE across both datasets (0.038 ± 0.028 on EMOB, 0.490 ± 0.172 on appliances energy) is consistent with its primary optimization objective of reconstruction fidelity. However, its utility retention results reveal, that it is highly dependent on focusing on the relevant signals.

The TCN-RNN baseline demonstrates that continuous latent representations become increasingly difficult at high compression ratios. Operating at only 3.0x compression with a correlation of 0.726 on EMOB, which is substantially below the 200x+ compression of discrete methods, it shows that continuous bottleneck compression is inherently limited for sequence compression tasks. To be competitive, the architecture of the TCN-RNN baseline would need to be significantly more complex, making the model heavier. While discrete latent representations provide a clear advantage, the baseline results also underscore that model objectives matter significantly. The superior performance of the discrete methods stems not only from discrete latents, but also from the specific optimization objectives designed for each architecture.

The relationship between reconstruction quality and downstream utility retention exhibits important insights into the relationship of the different metrics used to measure the performance of the neural compression methods. For the EMOB anomaly detection task, reconstruction metrics serve as reliable predictors of utility retention. This might be, because the task fundamentally depends on capturing sparse, discontinuous events in the signal. When these critical events are not reliably reconstructed, as evidenced by poor MSE, downstream anomaly detection inherently suffers. This would explain the correlation visible in Table 5.3 between reconstruction metrics and utility retention for the EMOB dataset. In contrast, the appliances energy data has more signals and therefore also more noise in the downstream task. Not all of these signals are likely to correlate strongly with the regression target, making the overall reconstruction quality less meaningful as a predictor for donwstream task utility retention, and signal-specific metrics more relevant. This task-dependent behavior highlights that compression suitability cannot be assessed universally through reconstruction metrics alone. It is vital that the choice of compression method be informed by the specific requirements of the downstream application, and that compression methods are specifically evaluated using both reconstruction quality and utility retention metrics.

7

Conclusion

Two important phenomena are highlighted when summarizing the experimental results and taking possible explanations into account. First, continuous latent autoencoders, at least those with deterministic encoders, do not have the capacity to achieve satisfactory multimodal compression results while remaining lightweight enough to be edge-compatible. The TCN-RNN model is fundamentally constrained in its capabilities to achieve competitive CRs, and even at much lower CRs it cannot compete with discrete-latent neural compression models in terms of reconstruction quality.

The second important takeaway is that, when constructing neural compression methods, it is vital to be conscious of the optimization landscape. The experiments make clear that the limits of representational capacity for edge-optimized neural compression methods are real and must be accounted for. When constraining the optimization landscape, for example through the introduction of a partitioned codebook, stability can be gained. This is indicated by the stability observed in the UniEdgeCodec model. But importantly, this stability comes with a trade-off. When loosening the optimization landscape, for example by adding a task loss term or a shared codebook, models become less reliable but also less constrained. The higher peak utility retention of both TAEdgeCodec and EdgeCodec and the signal specific superior reconstruction performance shows that these models benefit from this freedom, whereas UniEdgeCodec is more constrained. The freedom to be able to not only learn correlating signal patterns in a shared optimization landscape, but also assign these correlating patterns to a shared codebook, can improve the reconstruction of correlated patterns. For neural compression methods that produce data for downstream model training, this means that being conscious of how the downstream task goals and the training goals align is vital. It is important to one be aware of the importance of specific signals for the downstream tasks, and two, take the trade-off between stability and model freedom into account. It has further been shown, that a holistic evaluation including both reconstruction metrics and utility retention metrics is need to fully understand the efficacy of neural compression methods that provide compressed data for downstream model training.

Bibliography

[1] E. A. Lee and S. A. Seshia, *Introduction to Embedded Systems: A Cyber-Physical Systems Approach*, 2nd. Cambridge, MA, USA: MIT Press, 2017, Available at `https://ptolemy.berkeley.edu/books/leeseshia/`.

[2] N. Navet and F. Simonot-Lion, Eds., *Automotive Embedded Systems Handbook*. Boca Raton, FL, USA: CRC Press, 2009, ISBN: 978-0-8493-8026-6.

[3] M. Corporation, "Telemetry data: A resource for automotive safety insights," MITRE, Tech. Rep. PR-23-2484, 2023, `https://www.mitre.org/sites/default/files/2023-08/PR-23-2484-Telemetry-Data-A-Resource-for-Automotive-Safety-Insights.pdf`.

[4] A. Consortium, "Specification of diagnostic log and trace," AUTOSAR, Tech. Rep. R22-11, 2022, `https://www.autosar.org/fileadmin/standards/R22-11/CP/AUTOSAR_SWS_DiagnosticLogAndTrace.pdf`.

[5] M. Clemens, T. Müller, and R. Schäfer, "Event-driven fault diagnosis framework for automotive systems," US20110238258A1, U.S. Patent Application Publication, Sep. 29, 2011. [Online]. Available: `https://patents.google.com/patent/US20110238258A1/en`.

[6] M. Bertoncello, C. Martens, T. Möller, and T. Schneiderbauer, "Unlocking the full life-cycle value from connected-car data," McKinsey & Company, McKinsey Center for Future Mobility, Tech. Rep., Feb. 2021, White paper. [Online]. Available: `https://www.mckinsey.com/industries/automotive-and-assembly/our-insights/unlocking-the-full-life-cycle-value-from-connected-car-data`.

[7] R. Samantaray, "Adas sensor data handling in the world of autonomous mobility," *SAE Technical Paper Series*, 2023. DOI: `10.4271/2023-01-0993`.

[8] A. Theissler, J. Pérez-Velázquez, M. Kettelgerdes, and G. Elger, "Predictive maintenance enabled by machine learning: Use cases and challenges in the automotive industry," *Reliability Engineering & System Safety*, vol. 215, p. 107 864, 2021, ISSN: 0951-8320. DOI: `https://doi.org/10.1016/j.ress.2021.107864`. [Online]. Available: `https://www.sciencedirect.com/science/article/pii/S0951832021003835`.

[9] Ö. Özdemir, M. T. İşyapar, P. Karagöz, K. W. Schmidt, D. Demir, and N. A. Karagöz, *A survey of anomaly detection in in-vehicle networks*, 2024. arXiv: `2409.07505 [cs.LG]`. [Online]. Available: `https://arxiv.org/abs/2409.07505`.

[10] P. H. Brunheroto, A. L. Gonçalves Pepino, F. Deschamps, and E. de Freitas Rocha Loures, "Data analytics in fleet operations: A systematic literature

review and workflow proposal," *Procedia CIRP*, vol. 107, pp. 1192–1197, 2022, Leading manufacturing systems transformation – Proceedings of the 55th CIRP Conference on Manufacturing Systems 2022, ISSN: 2212-8271. DOI: https://doi.org/10.1016/j.procir.2022.05.130. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2212827122004140.

[11] S. Tirupathi, D. Salwala, G. Zizzo, *et al.*, "Machine learning platform for extreme scale computing on compressed iot data," in *2022 IEEE International Conference on Big Data (Big Data)*, 2022, pp. 3179–3185. DOI: 10.1109/BigData55660.2022.10020540.

[12] F. Eichinger, P. Efros, S. Karnouskos, and K. Böhm, "A time-series compression technique and its application to the smart grid," *The VLDB journal*, vol. 24, no. 2, pp. 193–218, 2015, ISSN: 0949-877X, 1066-8888. DOI: 10.1007/s00778-014-0368-8.

[13] A. Moon, J. Kim, J. Zhang, and S. W. Son, "Evaluating fidelity of lossy compression on spatiotemporal data from an iot enabled smart farm," *Computers and Electronics in Agriculture*, vol. 154, pp. 304–313, 2018, ISSN: 0168-1699. DOI: https://doi.org/10.1016/j.compag.2018.08.045. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0168169918303697.

[14] C. E. Shannon, "A mathematical theory of communication," *Bell System Technical Journal*, vol. 27, no. 3, pp. 379–423, 1948. DOI: https://doi.org/10.1002/j.1538-7305.1948.tb01338.x. eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1002/j.1538-7305.1948.tb01338.x. [Online]. Available: https://onlinelibrary.wiley.com/doi/abs/10.1002/j.1538-7305.1948.tb01338.x.

[15] C. E. Shannon, "Communication in the presence of noise," *Proceedings of the I.R.E.*, vol. 37, no. 1, pp. 10–21, Jan. 1949. DOI: 10.1109/JRPROC.1949.232969.

[16] A. Marchioni, A. Enttsel, M. Mangia, R. Rovatti, and G. Setti, "Anomaly detection based on compressed data: An information theoretic characterization," *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, vol. 54, no. 1, pp. 23–38, 2024. DOI: 10.1109/TSMC.2023.10233222.

[17] C. E. Muniz-Cuza, S. K. Jensen, J. Brusokas, N. Ho, and T. B. Pedersen, "Evaluating the impact of error-bounded lossy compression on time series forecasting," in *Advances in Database Technology - EDBT*, ser. Advances in Database Technology - EDBT, OpenProceedings, Mar. 2024, pp. 650–663. DOI: 10.48786/edbt.2024.56.

[18] Y. Yang, S. Mandt, and L. Theis, "An introduction to neural data compression," *Found. Trends Comput. Graph. Vis.*, vol. 15, pp. 113–200, 2022. DOI: 10.1561/0600000107.

[19] N. Tishby, F. C. Pereira, and W. Bialek, *The information bottleneck method*, 2000. arXiv: physics/0004057 [physics.data-an]. [Online]. Available: https://arxiv.org/abs/physics/0004057.

[20] S. Chen and W. Guo, "Auto-encoders in deep learning—a review with new perspectives," *Mathematics*, vol. 11, no. 8, 2023, ISSN: 2227-7390. DOI: 10.

3390/math11081777. [Online]. Available: https://www.mdpi.com/2227-7390/11/8/1777.

[21] A. A. Alemi, I. Fischer, J. V. Dillon, and K. Murphy, *Deep variational information bottleneck*, 2016. arXiv: 1612.00410 [cs.LG]. [Online]. Available: https://arxiv.org/abs/1612.00410.

[22] Z. Zheng and Z. Zhang, "A temporal convolutional recurrent autoencoder based framework for compressing time series data," *Applied Soft Computing*, vol. 147, p. 110 797, 2023, ISSN: 1568-4946. DOI: https://doi.org/10.1016/j.asoc.2023.110797. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1568494623008153.

[23] J. Liu, P. Djukic, M. Kulhandjian, and B. Kantarci, *Deep dict: Deep learning-based lossy time series compressor for iot data*, 2024. arXiv: 2401.10396 [eess.SP]. [Online]. Available: https://arxiv.org/abs/2401.10396.

[24] B. Hodo, T. Polonelli, A. Moallemi, L. Benini, and M. Magno, "Edgecodec: Onboard lightweight high fidelity neural compressor with residual vector quantization," in *2025 10th International Workshop on Advances in Sensors and Interfaces (IWASI)*, 2025, pp. 1–6. DOI: 10.1109/IWASI66786.2025.11121988.

[25] J. Azar, A. Makhoul, R. Couturier, and J. Demerjian, "Robust iot time series classification with data compression and deep learning," *Neurocomputing*, vol. 398, Feb. 2020. DOI: 10.1016/j.neucom.2020.02.097.

[26] J. Löhdefink, A. Bär, N. M. Schmidt, F. Hüger, P. Schlicht, and T. Fingscheidt, "Gan- vs. jpeg2000 image compression for distributed automotive perception: Higher peak snr does not mean better semantic segmentation," *arXiv preprint arXiv:1902.04311*, 2019. DOI: arXiv:1902.04311v1.

[27] N. Zeghidour, A. Luebs, A. Omran, J. Skoglund, and M. Tagliasacchi, "Soundstream: An end-to-end neural audio codec," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 30, pp. 495–507, 2021.

[28] A. Défossez, J. Copet, G. Synnaeve, and Y. Adi, "High fidelity neural audio compression," *arXiv preprint arXiv:2210.13438*, 2022.

[29] S. Ji, Z. Jiang, W. Wang, *et al.*, *Wavtokenizer: An efficient acoustic discrete codec tokenizer for audio language modeling*, 2025. arXiv: 2408.16532 [eess.AS]. [Online]. Available: https://arxiv.org/abs/2408.16532.

[30] A. Consortium, "Layered software architecture," AUTOSAR, Tech. Rep. R22-11, 2022, https://www.autosar.org/fileadmin/standards/R22-11/CP/AUTOSAR_EXP_LayeredSoftwareArchitecture.pdf.

[31] M. Komorkiewicz, A. Chin, P. Skruch, and M. Szelest, "Intelligent data handling in current and next-generation automated vehicle development—a review," *IEEE Access*, vol. PP, 2023. DOI: 10.1109/ACCESS.2023.3258623.

[32] N. Lai, D. A. Dewi, S. S. Maidin, W. Xiao, S. Zhao, and Q. Hu, "A comprehensive review of lightweight deep learning models for edge computing with future directions," *Discover Computing*, vol. 29, p. 110, 2026. DOI: 10.1007/s10791-026-10021-3.

[33] K. Sayood, *Introduction to Data Compression*, 5th. Morgan Kaufmann Publishers Inc., 2017, ISBN: 978-0-12-809474-7.

[34] A. Thomasian, "Chapter 2 - storage technologies and their data," in *Storage Systems*, A. Thomasian, Ed., Morgan Kaufmann, 2022, pp. 89–196, ISBN: 978-0-

323-90796-5. DOI: https://doi.org/10.1016/B978-0-32-390796-5.00011-5. [Online]. Available: https://www.sciencedirect.com/science/article/pii/B9780323907965000115.

[35] T. M. Cover and J. A. Thomas, *Elements of Information Theory, 2nd Edition*, 2nd. Hoboken, NJ: John Wiley & Sons, 2006, ISBN: 978-0-471-24195-9.

[36] S. Ma, X. Zhang, C. Jia, Z. Zhao, S. Wang, and S. Wang, "Image and video compression with neural networks: A review," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 30, no. 6, pp. 1683–1698, Jun. 2020, ISSN: 1558-2205. DOI: 10.1109/tcsvt.2019.2910119. [Online]. Available: http://dx.doi.org/10.1109/TCSVT.2019.2910119.

[37] S. Johnsson, "Large scale efficient data readout for vehicle fleets," M.S. thesis, Chalmers University of Technology, 2025.

[38] N. Tishby and N. Zaslavsky, "Deep learning and the information bottleneck principle," in *2015 IEEE Information Theory Workshop (ITW)*, 2015, pp. 1–5. DOI: 10.1109/ITW.2015.7133169.

[39] R. Shwartz-Ziv and N. Tishby, "Opening the black box of deep neural networks via information," *arXiv preprint arXiv:1703.00810*, 2017.

[40] A. M. Saxe, Y. Bansal, J. Dapello, *et al.*, "On the information bottleneck theory of deep learning," in *International Conference on Learning Representations*, 2018. [Online]. Available: https://openreview.net/forum?id=ry_WPG-A-.

[41] Z. Goldfeld and Y. Polyanskiy, *The information bottleneck problem and its applications in machine learning*, 2020. arXiv: 2004.14941 [cs.LG]. [Online]. Available: https://arxiv.org/abs/2004.14941.

[42] D. P. Kingma and M. Welling, *Auto-encoding variational bayes*, 2013. arXiv: 1312.6114 [stat.ML]. [Online]. Available: https://arxiv.org/abs/1312.6114.

[43] L. Theis and E. Agustsson, "On the advantages of stochastic encoders," *CoRR*, vol. abs/2102.09270, 2021. arXiv: 2102.09270. [Online]. Available: https://arxiv.org/abs/2102.09270.

[44] D. Strouse and D. J. Schwab, *The deterministic information bottleneck*, 2016. arXiv: 1604.00268 [q-bio.NC]. [Online]. Available: https://arxiv.org/abs/1604.00268.

[45] S. S. Du, Y. Wang, X. Zhai, S. Balakrishnan, R. Salakhutdinov, and A. Singh, *How many samples are needed to estimate a convolutional or recurrent neural network?* 2019. arXiv: 1805.07883 [stat.ML]. [Online]. Available: https://arxiv.org/abs/1805.07883.

[46] N. Johnston, E. Eban, A. Gordon, and J. Ballé, *Computationally efficient neural image compression*, 2019. arXiv: 1912.08771 [eess.IV]. [Online]. Available: https://arxiv.org/abs/1912.08771.

[47] B. Hodo, T. Polonelli, A. Moallemi, L. Benini, and M. Magno. "Edgecodec." GitHub repository. (2025), [Online]. Available: https://github.com/ETH-PBL/EdgeCodec (visited on 04/25/2026).

[48] P. Wang. "Vector-quantize-pytorch." GitHub repository. (2026), [Online]. Available: https://github.com/lucidrains/vector-quantize-pytorch (visited on 04/25/2026).

[49] Y. Jiang, Q. Chen, S. Ji, *et al.*, *Unicodec: Unified audio codec with single domain-adaptive codebook*, 2025. arXiv: 2502.20067 [eess.AS]. [Online]. Available: https://arxiv.org/abs/2502.20067.

[50] L. I. Rudin, S. Osher, and E. Fatemi, "Nonlinear total variation based noise removal algorithms," *Physica D: Nonlinear Phenomena*, vol. 60, no. 1, pp. 259–268, 1992, ISSN: 0167-2789. DOI: https://doi.org/10.1016/0167-2789(92)90242-F. [Online]. Available: https://www.sciencedirect.com/science/article/pii/016727899290242F.

[51] J. Liu, Y. Sun, X. Xu, and U. S. Kamilov, *Image restoration using total variation regularized deep image prior*, 2018. arXiv: 1810.12864 [cs.CV]. [Online]. Available: https://arxiv.org/abs/1810.12864.

[52] G. G. Pihlgren, K. Nikolaidou, P. C. Chhipa, *et al.*, *A systematic performance analysis of deep perceptual loss networks: Breaking transfer learning conventions*, 2024. arXiv: 2302.04032 [cs.CV]. [Online]. Available: https://arxiv.org/abs/2302.04032.

[53] M. Song, J. Choi, and B. Han, *Variable-rate deep image compression through spatially-adaptive feature transform*, 2021. arXiv: 2108.09551 [eess.IV]. [Online]. Available: https://arxiv.org/abs/2108.09551.

[54] L. Candanedo, *Appliances Energy Prediction*, UCI Machine Learning Repository, DOI: https://doi.org/10.24432/C5VC8G, 2017.

[55] K.-J. Stol and B. Fitzgerald, "The abc of software engineering research," *ACM Trans. Softw. Eng. Methodol.*, vol. 27, no. 3, Sep. 2018, ISSN: 1049-331X. DOI: 10.1145/3241743. [Online]. Available: https://doi.org/10.1145/3241743.

A

Appendix

Complete Empirical Results

This appendix contains the complete empirical results for all models and datasets. Each section presents the detailed metrics for a specific model, including reconstruction performance, compression analysis, and downstream task utility results.

UniEdgeCodec

EMOB

Seed 42

```
============================================================
DECODER LOSS ANALYSIS
============================================================

Normalized MSE (summed): 0.072204
Normalized MSE (meaned): 0.024068
Correlation (meaned): 0.962822

Per-Channel Metrics:
------------------------------------------------------------

Amps:
  signal_variance: 0.052480
  signal_std: 0.229085
  autocorrelation: 0.989021
  mse_normalized: 0.007993
  rmse_normalized: 0.089404
  mae_normalized: 0.055657
  correlation: 0.920909
  mse_original: 0.736346
  rmse_original: 0.858106
  rmse_pct_std: 8.940404
  mape_pct: 299391316.370441
```

```
Volts:
  signal_variance: 1.007425
  signal_std: 1.003706
  autocorrelation: 0.999956
  mse_normalized: 0.014074
  rmse_normalized: 0.118635
  mae_normalized: 0.088089
  correlation: 0.993049
  mse_original: 0.001110
  rmse_original: 0.033320
  rmse_pct_std: 11.863457
  mape_pct: 0.659826

Temp 1:
  signal_variance: 0.875158
  signal_std: 0.935499
  autocorrelation: 0.997486
  mse_normalized: 0.050137
  rmse_normalized: 0.223913
  mae_normalized: 0.156248
  correlation: 0.974507
  mse_original: 2.799337
  rmse_original: 1.673122
  rmse_pct_std: 22.391304
  mape_pct: 8.629893

============================================================
COMPRESSION ANALYSIS
============================================================

Model Type: uni_edgecodec

Compression Metrics:
  Average codes per window: 6.00
  Bits per code: 10.0
  Average bits per window: 59.8
  Average bits per timestep: 0.467
  Compression ratio: 205.5x

============================================================
DOWNSTREAM TASK UTILITY
============================================================

Task: Battery Anomaly Detection
Task Type: classification
```

```
Samples: 28274

Original Signal Performance:
  accuracy: 0.866136
  precision: 0.125000
  recall: 0.384279
  f1: 0.188639
  auc: 0.735684

Reconstructed Signal Performance:
  accuracy: 0.815915
  precision: 0.073529
  recall: 0.305677
  f1: 0.118544
  auc: 0.630800

Utility Retention: 62.85%
```

Seed 100

```
============================================================
DECODER LOSS ANALYSIS
============================================================

Normalized MSE (summed): 0.080150
Normalized MSE (meaned): 0.026717
Correlation (meaned): 0.965547

Per-Channel Metrics:
------------------------------------------------------------

Amps:
  signal_variance: 0.052480
  signal_std: 0.229085
  autocorrelation: 0.989021
  mse_normalized: 0.006668
  rmse_normalized: 0.081657
  mae_normalized: 0.047874
  correlation: 0.935477
  mse_original: 0.614259
  rmse_original: 0.783746
  rmse_pct_std: 8.165668
  mape_pct: 218366485.355352

Volts:
  signal_variance: 1.007425
```

```
  signal_std: 1.003706
  autocorrelation: 0.999956
  mse_normalized: 0.041661
  rmse_normalized: 0.204111
  mae_normalized: 0.158830
  correlation: 0.979121
  mse_original: 0.003286
  rmse_original: 0.057327
  rmse_pct_std: 20.411054
  mape_pct: 1.207389

Temp 1:
  signal_variance: 0.875158
  signal_std: 0.935499
  autocorrelation: 0.997486
  mse_normalized: 0.031821
  rmse_normalized: 0.178384
  mae_normalized: 0.123773
  correlation: 0.982042
  mse_original: 1.776678
  rmse_original: 1.332921
  rmse_pct_std: 17.838413
  mape_pct: 7.195364

============================================================
COMPRESSION ANALYSIS
============================================================

Model Type: uni_edgecodec

Compression Metrics:
  Average codes per window: 6.00
  Bits per code: 10.0
  Average bits per window: 59.8
  Average bits per timestep: 0.467
  Compression ratio: 205.5x

============================================================
DOWNSTREAM TASK UTILITY
============================================================

Task: Battery Anomaly Detection
Task Type: classification
Samples: 28274

Original Signal Performance:
```

```
  accuracy: 0.866136
  precision: 0.125000
  recall: 0.384279
  f1: 0.188639
  auc: 0.735684

Reconstructed Signal Performance:
  accuracy: 0.778957
  precision: 0.072087
  recall: 0.375546
  f1: 0.120956
  auc: 0.631253

Utility Retention: 64.15%
```

Seed 200

```
============================================================
DECODER LOSS ANALYSIS
============================================================

Normalized MSE (summed): 0.119177
Normalized MSE (meaned): 0.039726
Correlation (meaned): 0.953058

Per-Channel Metrics:
------------------------------------------------------------

Amps:
  signal_variance: 0.052480
  signal_std: 0.229085
  autocorrelation: 0.989021
  mse_normalized: 0.008403
  rmse_normalized: 0.091668
  mae_normalized: 0.062466
  correlation: 0.917116
  mse_original: 0.774105
  rmse_original: 0.879832
  rmse_pct_std: 9.166765
  mape_pct: 242180621.590186

Volts:
  signal_variance: 1.007425
  signal_std: 1.003706
  autocorrelation: 0.999956
  mse_normalized: 0.022721
```

```
  rmse_normalized: 0.150735
  mae_normalized: 0.113880
  correlation: 0.988787
  mse_original: 0.001792
  rmse_original: 0.042336
  rmse_pct_std: 15.073479
  mape_pct: 0.861688

Temp 1:
  signal_variance: 0.875158
  signal_std: 0.935499
  autocorrelation: 0.997486
  mse_normalized: 0.088053
  rmse_normalized: 0.296738
  mae_normalized: 0.198938
  correlation: 0.953270
  mse_original: 4.916341
  rmse_original: 2.217282
  rmse_pct_std: 29.673774
  mape_pct: 9.289663

============================================================
COMPRESSION ANALYSIS
============================================================

Model Type: uni_edgecodec

Compression Metrics:
  Average codes per window: 6.00
  Bits per code: 10.0
  Average bits per window: 59.8
  Average bits per timestep: 0.467
  Compression ratio: 205.5x

============================================================
DOWNSTREAM TASK UTILITY
============================================================

Task: Battery Anomaly Detection
Task Type: classification
Samples: 28274

Original Signal Performance:
  accuracy: 0.866136
  precision: 0.125000
  recall: 0.384279
```

```
  f1: 0.188639
  auc: 0.735684

Reconstructed Signal Performance:
  accuracy: 0.731565
  precision: 0.070047
  recall: 0.458515
  f1: 0.121528
  auc: 0.644242

Utility Retention: 64.43%
```

Smart Home

Seed 42

```
============================================================
DECODER LOSS ANALYSIS
============================================================

Normalized MSE (summed): 3.271303
Normalized MSE (meaned): 0.467329
Correlation (meaned): 0.677110
Normalized MSE (meaned, no lights): 0.510770
Correlation (meaned, no lights): 0.767588

Per-Channel Metrics:
------------------------------------------------------------

lights:
  signal_variance: 0.192364
  signal_std: 0.438593
  autocorrelation: 0.999889
  mse_normalized: 0.206683
  rmse_normalized: 0.454624
  mae_normalized: 0.379427
  correlation: 0.134240
  mse_original: 14.327820
  rmse_original: 3.785211
  rmse_pct_std: 45.462399
  mape_pct: 204.203971

Press_mm_hg:
  signal_variance: 1.466955
  signal_std: 1.211179
  autocorrelation: 0.998368
```

```
  mse_normalized: 0.306090
  rmse_normalized: 0.553254
  mae_normalized: 0.450821
  correlation: 0.893336
  mse_original: 18.523853
  rmse_original: 4.303935
  rmse_pct_std: 55.325402
  mape_pct: 0.466526

T_out:
  signal_variance: 0.928853
  signal_std: 0.963770
  autocorrelation: 0.999245
  mse_normalized: 0.856698
  rmse_normalized: 0.925580
  mae_normalized: 0.779588
  correlation: 0.833046
  mse_original: 14.571287
  rmse_original: 3.817235
  rmse_pct_std: 92.557980
  mape_pct: 24.392858

RH_out:
  signal_variance: 0.972808
  signal_std: 0.986310
  autocorrelation: 0.999397
  mse_normalized: 1.036181
  rmse_normalized: 1.017930
  mae_normalized: 0.866260
  correlation: 0.746812
  mse_original: 174.611465
  rmse_original: 13.214063
  rmse_pct_std: 101.792995
  mape_pct: 10.489674

Windspeed:
  signal_variance: 0.684800
  signal_std: 0.827527
  autocorrelation: 0.984950
  mse_normalized: 0.326861
  rmse_normalized: 0.571718
  mae_normalized: 0.422726
  correlation: 0.725791
  mse_original: 2.122498
  rmse_original: 1.456880
  rmse_pct_std: 57.171801
```

```
  mape_pct: 46.745505

Visibility:
  signal_variance: 0.479381
  signal_std: 0.692374
  autocorrelation: 0.996119
  mse_normalized: 0.173289
  rmse_normalized: 0.416280
  mae_normalized: 0.342659
  correlation: 0.807569
  mse_original: 26.524228
  rmse_original: 5.150168
  rmse_pct_std: 41.627992
  mape_pct: 13.582430

Tdewpoint:
  signal_variance: 0.532463
  signal_std: 0.729701
  autocorrelation: 0.774134
  mse_normalized: 0.365500
  rmse_normalized: 0.604566
  mae_normalized: 0.364910
  correlation: 0.598975
  mse_original: 4.486677
  rmse_original: 2.118178
  rmse_pct_std: 60.456631
  mape_pct: 79.207030

============================================================
COMPRESSION ANALYSIS
============================================================

Model Type: uni_edgecodec

Compression Metrics:
  Average codes per window: 14.00
  Bits per code: 10.0
  Average bits per window: 139.5
  Average bits per timestep: 0.969
  Compression ratio: 231.2x

============================================================
DOWNSTREAM TASK UTILITY
============================================================

Task: Appliance Energy Sequence Prediction
```

```
Task Type: regression
Samples: 1831

Original Signal Performance:
  mse: 3424.935426
  mae: 27.434254
  r2: 0.539275
  rmse: 58.522948
  per_timestep_r2_mean: 0.539275
  per_timestep_r2_min: 0.283651
  per_timestep_r2_max: 0.683567
  per_timestep_corr_mean: 0.742444
  per_timestep_corr_min: 0.558071
  target_sequence_length: 144.000000

Reconstructed Signal Performance:
  mse: 4084.952836
  mae: 30.917958
  r2: 0.451651
  rmse: 63.913636
  per_timestep_r2_mean: 0.451651
  per_timestep_r2_min: 0.237909
  per_timestep_r2_max: 0.603839
  per_timestep_corr_mean: 0.684367
  per_timestep_corr_min: 0.499326
  target_sequence_length: 144.000000

Utility Retention: 83.75%
```

Seed 100

```
============================================================
DECODER LOSS ANALYSIS
============================================================

Normalized MSE (summed): 5.379095

Per-Channel Metrics:
------------------------------------------------------------

lights:
  signal_variance: 0.192364
  signal_std: 0.438593
  autocorrelation: 0.999889
  mse_normalized: 0.226110
  rmse_normalized: 0.475510
```

```
  mae_normalized: 0.404550
  correlation: 0.175448
  mse_original: 15.674522
  rmse_original: 3.959106
  rmse_pct_std: 47.550976
  mape_pct: 159.842583

Press_mm_hg:
  signal_variance: 1.466955
  signal_std: 1.211179
  autocorrelation: 0.998368
  mse_normalized: 0.338650
  rmse_normalized: 0.581936
  mae_normalized: 0.478472
  correlation: 0.891780
  mse_original: 20.494301
  rmse_original: 4.527063
  rmse_pct_std: 58.193632
  mape_pct: 0.495218

T_out:
  signal_variance: 0.928853
  signal_std: 0.963770
  autocorrelation: 0.999245
  mse_normalized: 1.525240
  rmse_normalized: 1.235006
  mae_normalized: 1.074025
  correlation: 0.743808
  mse_original: 25.942299
  rmse_original: 5.093358
  rmse_pct_std: 123.500621
  mape_pct: 33.366096

RH_out:
  signal_variance: 0.972808
  signal_std: 0.986310
  autocorrelation: 0.999397
  mse_normalized: 2.327205
  rmse_normalized: 1.525518
  mae_normalized: 1.342499
  correlation: 0.520749
  mse_original: 392.167515
  rmse_original: 19.803220
  rmse_pct_std: 152.551795
  mape_pct: 16.141582
```

```
Windspeed:
  signal_variance: 0.684800
  signal_std: 0.827527
  autocorrelation: 0.984950
  mse_normalized: 0.283938
  rmse_normalized: 0.532858
  mae_normalized: 0.411283
  correlation: 0.789689
  mse_original: 1.843769
  rmse_original: 1.357854
  rmse_pct_std: 53.285795
  mape_pct: 54.741608

Visibility:
  signal_variance: 0.479381
  signal_std: 0.692374
  autocorrelation: 0.996119
  mse_normalized: 0.310095
  rmse_normalized: 0.556862
  mae_normalized: 0.449996
  correlation: 0.764877
  mse_original: 47.464291
  rmse_original: 6.889433
  rmse_pct_std: 55.686200
  mape_pct: 19.071479

Tdewpoint:
  signal_variance: 0.532463
  signal_std: 0.729701
  autocorrelation: 0.774134
  mse_normalized: 0.367857
  rmse_normalized: 0.606512
  mae_normalized: 0.467391
  correlation: 0.641483
  mse_original: 4.515607
  rmse_original: 2.124996
  rmse_pct_std: 60.651228
  mape_pct: 128.866373

============================================================
COMPRESSION ANALYSIS
============================================================

Model Type: uni_edgecodec

Compression Metrics:
```

```
  Average codes per window: 14.00
  Bits per code: 10.0
  Average bits per window: 139.5
  Average bits per timestep: 0.969
  Compression ratio: 231.2x

============================================================
DOWNSTREAM TASK UTILITY
============================================================

Task: Appliance Energy Sequence Prediction
Task Type: regression
Samples: 1831

Original Signal Performance:
  mse: 3424.935426
  mae: 27.434254
  r2: 0.539275
  rmse: 58.522948
  per_timestep_r2_mean: 0.539275
  per_timestep_r2_min: 0.283651
  per_timestep_r2_max: 0.683567
  per_timestep_corr_mean: 0.742444
  per_timestep_corr_min: 0.558071
  target_sequence_length: 144.000000

Reconstructed Signal Performance:
  mse: 4081.353037
  mae: 31.392059
  r2: 0.452205
  rmse: 63.885468
  per_timestep_r2_mean: 0.452205
  per_timestep_r2_min: 0.265192
  per_timestep_r2_max: 0.597965
  per_timestep_corr_mean: 0.684119
  per_timestep_corr_min: 0.538841
  target_sequence_length: 144.000000

Utility Retention: 83.85%
```

Seed 200

```
============================================================
DECODER LOSS ANALYSIS
============================================================
```

```
Normalized MSE (summed): 4.176516

Per-Channel Metrics:
------------------------------------------------------------

lights:
  signal_variance: 0.192364
  signal_std: 0.438593
  autocorrelation: 0.999889
  mse_normalized: 0.194077
  rmse_normalized: 0.440542
  mae_normalized: 0.380741
  correlation: 0.430296
  mse_original: 13.453975
  rmse_original: 3.667966
  rmse_pct_std: 44.054227
  mape_pct: 179.239094

Press_mm_hg:
  signal_variance: 1.466955
  signal_std: 1.211179
  autocorrelation: 0.998368
  mse_normalized: 0.497591
  rmse_normalized: 0.705401
  mae_normalized: 0.553710
  correlation: 0.829853
  mse_original: 30.113058
  rmse_original: 5.487537
  rmse_pct_std: 70.540144
  mape_pct: 0.574316

T_out:
  signal_variance: 0.928853
  signal_std: 0.963770
  autocorrelation: 0.999245
  mse_normalized: 1.034221
  rmse_normalized: 1.016967
  mae_normalized: 0.887095
  correlation: 0.841428
  mse_original: 17.590714
  rmse_original: 4.194129
  rmse_pct_std: 101.696651
  mape_pct: 27.614651

RH_out:
  signal_variance: 0.972808
```

```
  signal_std: 0.986310
  autocorrelation: 0.999397
  mse_normalized: 1.220779
  rmse_normalized: 1.104889
  mae_normalized: 0.940179
  correlation: 0.720779
  mse_original: 205.718777
  rmse_original: 14.342900
  rmse_pct_std: 110.488857
  mape_pct: 11.340527

Windspeed:
  signal_variance: 0.684800
  signal_std: 0.827527
  autocorrelation: 0.984950
  mse_normalized: 0.451774
  rmse_normalized: 0.672141
  mae_normalized: 0.528257
  correlation: 0.603383
  mse_original: 2.933624
  rmse_original: 1.712783
  rmse_pct_std: 67.214110
  mape_pct: 70.044661

Visibility:
  signal_variance: 0.479381
  signal_std: 0.692374
  autocorrelation: 0.996119
  mse_normalized: 0.437391
  rmse_normalized: 0.661355
  mae_normalized: 0.522254
  correlation: 0.618279
  mse_original: 66.948624
  rmse_original: 8.182214
  rmse_pct_std: 66.135541
  mape_pct: 22.306260

Tdewpoint:
  signal_variance: 0.532463
  signal_std: 0.729701
  autocorrelation: 0.774134
  mse_normalized: 0.340683
  rmse_normalized: 0.583681
  mae_normalized: 0.409055
  correlation: 0.634156
  mse_original: 4.182032
```

```
  rmse_original: 2.045002
  rmse_pct_std: 58.368053
  mape_pct: 107.928881

============================================================
COMPRESSION ANALYSIS
============================================================

Model Type: uni_edgecodec

Compression Metrics:
  Average codes per window: 14.00
  Bits per code: 10.0
  Average bits per window: 139.5
  Average bits per timestep: 0.969
  Compression ratio: 231.2x

============================================================
DOWNSTREAM TASK UTILITY
============================================================

Task: Appliance Energy Sequence Prediction
Task Type: regression
Samples: 1831

Original Signal Performance:
  mse: 3424.935426
  mae: 27.434254
  r2: 0.539275
  rmse: 58.522948
  per_timestep_r2_mean: 0.539275
  per_timestep_r2_min: 0.283651
  per_timestep_r2_max: 0.683567
  per_timestep_corr_mean: 0.742444
  per_timestep_corr_min: 0.558071
  target_sequence_length: 144.000000

Reconstructed Signal Performance:
  mse: 4065.300074
  mae: 30.573627
  r2: 0.451683
  rmse: 63.759706
  per_timestep_r2_mean: 0.451683
  per_timestep_r2_min: 0.032049
  per_timestep_r2_max: 0.585392
  per_timestep_corr_mean: 0.682978
```

```
  per_timestep_corr_min: 0.401906
  target_sequence_length: 144.000000

Utility Retention: 83.76%
```

TAEdgeCodec

EMOB

Seed 42

```
============================================================
DECODER LOSS ANALYSIS
============================================================

Normalized MSE (summed): 0.063650
Normalized MSE (meaned): 0.021217
Correlation (meaned): 0.771536

Per-Channel Metrics:
------------------------------------------------------------

Amps:
  signal_variance: 0.052480
  signal_std: 0.229085
  autocorrelation: 0.989021
  mse_normalized: 0.047063
  rmse_normalized: 0.216941
  mae_normalized: 0.170536
  correlation: 0.322999
  mse_original: 4.335604
  rmse_original: 2.082211
  rmse_pct_std: 21.694064
  mape_pct: 602169433.830990

Volts:
  signal_variance: 1.007425
  signal_std: 1.003706
  autocorrelation: 0.999956
  mse_normalized: 0.007051
  rmse_normalized: 0.083973
  mae_normalized: 0.058369
  correlation: 0.997072
  mse_original: 0.000556
  rmse_original: 0.023585
```

```
  rmse_pct_std: 8.397291
  mape_pct: 0.431633

Temp 1:
  signal_variance: 0.875158
  signal_std: 0.935499
  autocorrelation: 0.997486
  mse_normalized: 0.009536
  rmse_normalized: 0.097651
  mae_normalized: 0.063833
  correlation: 0.994537
  mse_original: 0.532418
  rmse_original: 0.729670
  rmse_pct_std: 9.765137
  mape_pct: 3.024229

============================================================
COMPRESSION ANALYSIS
============================================================

Model Type: edgecodec

Compression Metrics:
  Average codes per window: 6.00
  Bits per code: 9.6
  Average bits per window: 57.5
  Average bits per timestep: 0.449
  Compression ratio: 213.7x

============================================================
DOWNSTREAM TASK UTILITY
============================================================

Task: Battery Anomaly Detection
Task Type: classification
Samples: 28274

Original Signal Performance:
  accuracy: 0.866136
  precision: 0.125000
  recall: 0.384279
  f1: 0.188639
  auc: 0.735684

Reconstructed Signal Performance:
  accuracy: 0.800177
```

```
  precision: 0.084026
  recall: 0.397380
  f1: 0.138720
  auc: 0.668762

Utility Retention: 73.55%
```

Seed 100

```
============================================================
DECODER LOSS ANALYSIS
============================================================

Normalized MSE (summed): 0.303565
Normalized MSE (meaned): 0.101188
Correlation (meaned): 0.726986

Per-Channel Metrics:
------------------------------------------------------------

Amps:
  signal_variance: 0.052480
  signal_std: 0.229085
  autocorrelation: 0.989021
  mse_normalized: 0.047413
  rmse_normalized: 0.217745
  mae_normalized: 0.170623
  correlation: 0.313888
  mse_original: 4.367828
  rmse_original: 2.089935
  rmse_pct_std: 21.774535
  mape_pct: 537924826.578505

Volts:
  signal_variance: 1.007425
  signal_std: 1.003706
  autocorrelation: 0.999956
  mse_normalized: 0.053763
  rmse_normalized: 0.231868
  mae_normalized: 0.195146
  correlation: 0.974002
  mse_original: 0.004241
  rmse_original: 0.065123
  rmse_pct_std: 23.186849
  mape_pct: 1.491065
```

```
Temp 1:
  signal_variance: 0.875158
  signal_std: 0.935499
  autocorrelation: 0.997486
  mse_normalized: 0.202389
  rmse_normalized: 0.449877
  mae_normalized: 0.202214
  correlation: 0.893067
  mse_original: 11.300141
  rmse_original: 3.361568
  rmse_pct_std: 44.987694
  mape_pct: 12.030680

============================================================
COMPRESSION ANALYSIS
============================================================

Model Type: edgecodec

Compression Metrics:
  Average codes per window: 6.00
  Bits per code: 9.6
  Average bits per window: 57.5
  Average bits per timestep: 0.449
  Compression ratio: 213.7x

============================================================
DOWNSTREAM TASK UTILITY
============================================================

Task: Battery Anomaly Detection
Task Type: classification
Samples: 28274

Original Signal Performance:
  accuracy: 0.866136
  precision: 0.125000
  recall: 0.384279
  f1: 0.188639
  auc: 0.735684

Reconstructed Signal Performance:
  accuracy: 0.681698
  precision: 0.054453
  recall: 0.419214
  f1: 0.096386
```

```
  auc: 0.617036

Utility Retention: 51.08%
```

Seed 200

```
============================================================
DECODER LOSS ANALYSIS
============================================================

Normalized MSE (summed): 0.060148
Normalized MSE (meaned): 0.020049
Correlation (meaned): 0.795126

Per-Channel Metrics:
------------------------------------------------------------

Amps:
  signal_variance: 0.052480
  signal_std: 0.229085
  autocorrelation: 0.989021
  mse_normalized: 0.044357
  rmse_normalized: 0.210612
  mae_normalized: 0.163492
  correlation: 0.393797
  mse_original: 4.086339
  rmse_original: 2.021469
  rmse_pct_std: 21.061209
  mape_pct: 388906608.043791

Volts:
  signal_variance: 1.007425
  signal_std: 1.003706
  autocorrelation: 0.999956
  mse_normalized: 0.006115
  rmse_normalized: 0.078199
  mae_normalized: 0.050905
  correlation: 0.997016
  mse_original: 0.000482
  rmse_original: 0.021963
  rmse_pct_std: 7.819914
  mape_pct: 0.373749

Temp 1:
  signal_variance: 0.875158
  signal_std: 0.935499
```

```
  autocorrelation: 0.997486
  mse_normalized: 0.009676
  rmse_normalized: 0.098365
  mae_normalized: 0.061416
  correlation: 0.994566
  mse_original: 0.540228
  rmse_original: 0.735002
  rmse_pct_std: 9.836490
  mape_pct: 4.319947

============================================================
COMPRESSION ANALYSIS
============================================================

Model Type: edgecodec

Compression Metrics:
  Average codes per window: 6.00
  Bits per code: 9.6
  Average bits per window: 57.5
  Average bits per timestep: 0.449
  Compression ratio: 213.7x

============================================================
DOWNSTREAM TASK UTILITY
============================================================

Task: Battery Anomaly Detection
Task Type: classification
Samples: 28274

Original Signal Performance:
  accuracy: 0.866136
  precision: 0.125000
  recall: 0.384279
  f1: 0.188639
  auc: 0.735684

Reconstructed Signal Performance:
  accuracy: 0.787622
  precision: 0.086031
  recall: 0.441048
  f1: 0.143977
  auc: 0.664790

Utility Retention: 76.35%
```

Smart Home

Seed 42

```
============================================================
DECODER LOSS ANALYSIS
============================================================

Normalized MSE (summed): 4.228530
Normalized MSE (meaned): 0.604076
Correlation (meaned): 0.436676
Normalized MSE (meaned, no lights): 0.675328
Correlation (meaned, no lights): 0.451001

Per-Channel Metrics:
------------------------------------------------------------

lights:
  signal_variance: 0.192364
  signal_std: 0.438593
  autocorrelation: 0.999889
  mse_normalized: 0.176561
  rmse_normalized: 0.420192
  mae_normalized: 0.348169
  correlation: 0.350727
  mse_original: 12.239682
  rmse_original: 3.498526
  rmse_pct_std: 42.019159
  mape_pct: 230.033934

Press_mm_hg:
  signal_variance: 1.466955
  signal_std: 1.211179
  autocorrelation: 0.998368
  mse_normalized: 0.458521
  rmse_normalized: 0.677142
  mae_normalized: 0.579101
  correlation: 0.853607
  mse_original: 27.748610
  rmse_original: 5.267695
  rmse_pct_std: 67.714164
  mape_pct: 0.600269

T_out:
  signal_variance: 0.928853
```

```
  signal_std: 0.963770
  autocorrelation: 0.999245
  mse_normalized: 0.626130
  rmse_normalized: 0.791284
  mae_normalized: 0.633447
  correlation: 0.867734
  mse_original: 10.649640
  rmse_original: 3.263379
  rmse_pct_std: 79.128400
  mape_pct: 18.836082

RH_out:
  signal_variance: 0.972808
  signal_std: 0.986310
  autocorrelation: 0.999397
  mse_normalized: 0.740413
  rmse_normalized: 0.860472
  mae_normalized: 0.692568
  correlation: 0.827701
  mse_original: 124.770158
  rmse_original: 11.170056
  rmse_pct_std: 86.047227
  mape_pct: 8.333371

Windspeed:
  signal_variance: 0.684800
  signal_std: 0.827527
  autocorrelation: 0.984950
  mse_normalized: 0.786943
  rmse_normalized: 0.887098
  mae_normalized: 0.676620
  correlation: -0.125206
  mse_original: 5.110068
  rmse_original: 2.260546
  rmse_pct_std: 88.709793
  mape_pct: 73.397010

Visibility:
  signal_variance: 0.479381
  signal_std: 0.692374
  autocorrelation: 0.996119
  mse_normalized: 0.826320
  rmse_normalized: 0.909022
  mae_normalized: 0.786437
  correlation: 0.308947
  mse_original: 126.479505
```

```
  rmse_original: 11.246311
  rmse_pct_std: 90.902151
  mape_pct: 33.753118

Tdewpoint:
  signal_variance: 0.532463
  signal_std: 0.729701
  autocorrelation: 0.774134
  mse_normalized: 0.613643
  rmse_normalized: 0.783353
  mae_normalized: 0.589047
  correlation: -0.026775
  mse_original: 7.532730
  rmse_original: 2.744582
  rmse_pct_std: 78.335341
  mape_pct: 152.741428

============================================================
COMPRESSION ANALYSIS
============================================================

Model Type: edgecodec

Compression Metrics:
  Average codes per window: 14.00
  Bits per code: 9.6
  Average bits per window: 134.2
  Average bits per timestep: 0.932
  Compression ratio: 240.4x

============================================================
DOWNSTREAM TASK UTILITY
============================================================

Task: Appliance Energy Sequence Prediction
Task Type: regression
Samples: 1831

Original Signal Performance:
  mse: 3424.935426
  mae: 27.434254
  r2: 0.539275
  rmse: 58.522948
  per_timestep_r2_mean: 0.539275
  per_timestep_r2_min: 0.283651
  per_timestep_r2_max: 0.683567
```

```
  per_timestep_corr_mean: 0.742444
  per_timestep_corr_min: 0.558071
  target_sequence_length: 144.000000

Reconstructed Signal Performance:
  mse: 3819.401196
  mae: 29.233448
  r2: 0.485061
  rmse: 61.801304
  per_timestep_r2_mean: 0.485061
  per_timestep_r2_min: 0.249875
  per_timestep_r2_max: 0.708325
  per_timestep_corr_mean: 0.702363
  per_timestep_corr_min: 0.529794
  target_sequence_length: 144.000000

Utility Retention: 89.95%
```

Seed 100

```
============================================================
DECODER LOSS ANALYSIS
============================================================

Normalized MSE (summed): 3.409624

Per-Channel Metrics:
------------------------------------------------------------

lights:
  signal_variance: 0.192364
  signal_std: 0.438593
  autocorrelation: 0.999889
  mse_normalized: 0.205180
  rmse_normalized: 0.452968
  mae_normalized: 0.319623
  correlation: 0.371405
  mse_original: 14.223612
  rmse_original: 3.771420
  rmse_pct_std: 45.296770
  mape_pct: 239.298673

Press_mm_hg:
  signal_variance: 1.466955
  signal_std: 1.211179
  autocorrelation: 0.998368
```

```
  mse_normalized: 0.443606
  rmse_normalized: 0.666037
  mae_normalized: 0.540929
  correlation: 0.921397
  mse_original: 26.845983
  rmse_original: 5.181311
  rmse_pct_std: 66.603732
  mape_pct: 0.562717

T_out:
  signal_variance: 0.928853
  signal_std: 0.963770
  autocorrelation: 0.999245
  mse_normalized: 0.523456
  rmse_normalized: 0.723502
  mae_normalized: 0.579389
  correlation: 0.874313
  mse_original: 8.903282
  rmse_original: 2.983837
  rmse_pct_std: 72.350241
  mape_pct: 17.643389

RH_out:
  signal_variance: 0.972808
  signal_std: 0.986310
  autocorrelation: 0.999397
  mse_normalized: 0.659707
  rmse_normalized: 0.812223
  mae_normalized: 0.646720
  correlation: 0.749134
  mse_original: 111.170095
  rmse_original: 10.543723
  rmse_pct_std: 81.222341
  mape_pct: 7.814324

Windspeed:
  signal_variance: 0.684800
  signal_std: 0.827527
  autocorrelation: 0.984950
  mse_normalized: 0.534238
  rmse_normalized: 0.730916
  mae_normalized: 0.564402
  correlation: 0.509098
  mse_original: 3.469113
  rmse_original: 1.862556
  rmse_pct_std: 73.091600
```

```
  mape_pct: 71.590514

Visibility:
  signal_variance: 0.479381
  signal_std: 0.692374
  autocorrelation: 0.996119
  mse_normalized: 0.333234
  rmse_normalized: 0.577264
  mae_normalized: 0.488406
  correlation: 0.582215
  mse_original: 51.005958
  rmse_original: 7.141846
  rmse_pct_std: 57.726408
  mape_pct: 19.826833

Tdewpoint:
  signal_variance: 0.532463
  signal_std: 0.729701
  autocorrelation: 0.774134
  mse_normalized: 0.710204
  rmse_normalized: 0.842736
  mae_normalized: 0.704816
  correlation: -0.024359
  mse_original: 8.718064
  rmse_original: 2.952637
  rmse_pct_std: 84.273605
  mape_pct: 201.884974

============================================================
COMPRESSION ANALYSIS
============================================================

Model Type: edgecodec

Compression Metrics:
  Average codes per window: 14.00
  Bits per code: 9.6
  Average bits per window: 134.2
  Average bits per timestep: 0.932
  Compression ratio: 240.4x

============================================================
DOWNSTREAM TASK UTILITY
============================================================

Task: Appliance Energy Sequence Prediction
```

```
Task Type: regression
Samples: 1831

Original Signal Performance:
  mse: 3424.935426
  mae: 27.434254
  r2: 0.539275
  rmse: 58.522948
  per_timestep_r2_mean: 0.539275
  per_timestep_r2_min: 0.283651
  per_timestep_r2_max: 0.683567
  per_timestep_corr_mean: 0.742444
  per_timestep_corr_min: 0.558071
  target_sequence_length: 144.000000

Reconstructed Signal Performance:
  mse: 3836.404240
  mae: 29.300241
  r2: 0.481722
  rmse: 61.938714
  per_timestep_r2_mean: 0.481722
  per_timestep_r2_min: 0.228601
  per_timestep_r2_max: 0.648348
  per_timestep_corr_mean: 0.701853
  per_timestep_corr_min: 0.521868
  target_sequence_length: 144.000000

Utility Retention: 89.33%
```

Seed 200

```
============================================================
DECODER LOSS ANALYSIS
============================================================

Normalized MSE (summed): 3.658791

Per-Channel Metrics:
------------------------------------------------------------

lights:
  signal_variance: 0.192364
  signal_std: 0.438593
  autocorrelation: 0.999889
  mse_normalized: 0.101483
  rmse_normalized: 0.318564
  mae_normalized: 0.250448
```

```
  correlation: 0.696787
  mse_original: 7.035092
  rmse_original: 2.652375
  rmse_pct_std: 31.856435
  mape_pct: 146.226168

Press_mm_hg:
  signal_variance: 1.466955
  signal_std: 1.211179
  autocorrelation: 0.998368
  mse_normalized: 0.428357
  rmse_normalized: 0.654490
  mae_normalized: 0.542588
  correlation: 0.904595
  mse_original: 25.923161
  rmse_original: 5.091479
  rmse_pct_std: 65.448981
  mape_pct: 0.563484

T_out:
  signal_variance: 0.928853
  signal_std: 0.963770
  autocorrelation: 0.999245
  mse_normalized: 0.584328
  rmse_normalized: 0.764413
  mae_normalized: 0.583757
  correlation: 0.888341
  mse_original: 9.938634
  rmse_original: 3.152560
  rmse_pct_std: 76.441334
  mape_pct: 16.531099

RH_out:
  signal_variance: 0.972808
  signal_std: 0.986310
  autocorrelation: 0.999397
  mse_normalized: 0.847527
  rmse_normalized: 0.920612
  mae_normalized: 0.744979
  correlation: 0.818183
  mse_original: 142.820425
  rmse_original: 11.950750
  rmse_pct_std: 92.061209
  mape_pct: 8.914505

Windspeed:
```

```
  signal_variance: 0.684800
  signal_std: 0.827527
  autocorrelation: 0.984950
  mse_normalized: 0.731738
  rmse_normalized: 0.855417
  mae_normalized: 0.637552
  correlation: -0.129117
  mse_original: 4.751595
  rmse_original: 2.179815
  rmse_pct_std: 85.541710
  mape_pct: 84.948743

Visibility:
  signal_variance: 0.479381
  signal_std: 0.692374
  autocorrelation: 0.996119
  mse_normalized: 0.417164
  rmse_normalized: 0.645882
  mae_normalized: 0.541395
  correlation: 0.556712
  mse_original: 63.852615
  rmse_original: 7.990783
  rmse_pct_std: 64.588236
  mape_pct: 22.903054

Tdewpoint:
  signal_variance: 0.532463
  signal_std: 0.729701
  autocorrelation: 0.774134
  mse_normalized: 0.548194
  rmse_normalized: 0.740401
  mae_normalized: 0.538724
  correlation: 0.044171
  mse_original: 6.729320
  rmse_original: 2.594093
  rmse_pct_std: 74.040122
  mape_pct: 132.551740

============================================================
COMPRESSION ANALYSIS
============================================================

Model Type: edgecodec

Compression Metrics:
  Average codes per window: 14.00
```

```
  Bits per code: 9.6
  Average bits per window: 134.2
  Average bits per timestep: 0.932
  Compression ratio: 240.4x

============================================================
DOWNSTREAM TASK UTILITY
============================================================

Task: Appliance Energy Sequence Prediction
Task Type: regression
Samples: 1831

Original Signal Performance:
  mse: 3424.935426
  mae: 27.434254
  r2: 0.539275
  rmse: 58.522948
  per_timestep_r2_mean: 0.539275
  per_timestep_r2_min: 0.283651
  per_timestep_r2_max: 0.683567
  per_timestep_corr_mean: 0.742444
  per_timestep_corr_min: 0.558071
  target_sequence_length: 144.000000

Reconstructed Signal Performance:
  mse: 3859.931154
  mae: 29.783482
  r2: 0.479758
  rmse: 62.128344
  per_timestep_r2_mean: 0.479758
  per_timestep_r2_min: 0.201641
  per_timestep_r2_max: 0.681545
  per_timestep_corr_mean: 0.699960
  per_timestep_corr_min: 0.495231
  target_sequence_length: 144.000000

Utility Retention: 88.96%
```

EdgeCodec

EMOB

Seed 42

```
============================================================
DECODER LOSS ANALYSIS
============================================================

Normalized MSE (summed): 0.068077
Normalized MSE (meaned): 0.022692
Correlation (meaned): 0.771649

Per-Channel Metrics:
------------------------------------------------------------

Amps:
  signal_variance: 0.052480
  signal_std: 0.229085
  autocorrelation: 0.989021
  mse_normalized: 0.046928
  rmse_normalized: 0.216630
  mae_normalized: 0.171512
  correlation: 0.326028
  mse_original: 4.323187
  rmse_original: 2.079227
  rmse_pct_std: 21.662976
  mape_pct: 604023109.377981

Volts:
  signal_variance: 1.007425
  signal_std: 1.003706
  autocorrelation: 0.999956
  mse_normalized: 0.008071
  rmse_normalized: 0.089841
  mae_normalized: 0.060324
  correlation: 0.995989
  mse_original: 0.000637
  rmse_original: 0.025233
  rmse_pct_std: 8.984104
  mape_pct: 0.446543

Temp 1:
  signal_variance: 0.875158
  signal_std: 0.935499
  autocorrelation: 0.997486
```

```
  mse_normalized: 0.013078
  rmse_normalized: 0.114357
  mae_normalized: 0.079861
  correlation: 0.992930
  mse_original: 0.730172
  rmse_original: 0.854501
  rmse_pct_std: 11.435745
  mape_pct: 4.652572

============================================================
COMPRESSION ANALYSIS
============================================================

Model Type: edgecodec

Compression Metrics:
  Average codes per window: 6.00
  Bits per code: 9.6
  Average bits per window: 57.5
  Average bits per timestep: 0.449
  Compression ratio: 213.7x

============================================================
DOWNSTREAM TASK UTILITY
============================================================

Task: Battery Anomaly Detection
Task Type: classification
Samples: 28274

Original Signal Performance:
  accuracy: 0.866136
  precision: 0.125000
  recall: 0.384279
  f1: 0.188639
  auc: 0.735684

Reconstructed Signal Performance:
  accuracy: 0.797524
  precision: 0.081353
  recall: 0.388646
  f1: 0.134543
  auc: 0.627937

Utility Retention: 71.30%
```

Seed 100

```
============================================================
DECODER LOSS ANALYSIS
============================================================

Normalized MSE (summed): 0.212189
Normalized MSE (meaned): 0.070730
Correlation (meaned): 0.751513

Per-Channel Metrics:
------------------------------------------------------------

Amps:
  signal_variance: 0.052480
  signal_std: 0.229085
  autocorrelation: 0.989021
  mse_normalized: 0.046356
  rmse_normalized: 0.215305
  mae_normalized: 0.168475
  correlation: 0.341821
  mse_original: 4.270479
  rmse_original: 2.066514
  rmse_pct_std: 21.530515
  mape_pct: 513065546.827814

Volts:
  signal_variance: 1.007425
  signal_std: 1.003706
  autocorrelation: 0.999956
  mse_normalized: 0.040107
  rmse_normalized: 0.200268
  mae_normalized: 0.162644
  correlation: 0.979989
  mse_original: 0.003164
  rmse_original: 0.056248
  rmse_pct_std: 20.026769
  mape_pct: 1.236196

Temp 1:
  signal_variance: 0.875158
  signal_std: 0.935499
  autocorrelation: 0.997486
  mse_normalized: 0.125726
  rmse_normalized: 0.354579
  mae_normalized: 0.163713
  correlation: 0.932729
```

```
  mse_original: 7.019750
  rmse_original: 2.649481
  rmse_pct_std: 35.457868
  mape_pct: 10.679536

============================================================
COMPRESSION ANALYSIS
============================================================

Model Type: edgecodec

Compression Metrics:
  Average codes per window: 6.00
  Bits per code: 9.6
  Average bits per window: 57.5
  Average bits per timestep: 0.449
  Compression ratio: 213.7x

============================================================
DOWNSTREAM TASK UTILITY
============================================================

Task: Battery Anomaly Detection
Task Type: classification
Samples: 28274

Original Signal Performance:
  accuracy: 0.866136
  precision: 0.125000
  recall: 0.384279
  f1: 0.188639
  auc: 0.735684

Reconstructed Signal Performance:
  accuracy: 0.798055
  precision: 0.060635
  recall: 0.275109
  f1: 0.099369
  auc: 0.600173

Utility Retention: 52.69%
```

Seed 200

```
============================================================
DECODER LOSS ANALYSIS
```

```
============================================================

Normalized MSE (summed): 0.060925
Normalized MSE (meaned): 0.020308
Correlation (meaned): 0.816944

Per-Channel Metrics:
------------------------------------------------------------

Amps:
  signal_variance: 0.052480
  signal_std: 0.229085
  autocorrelation: 0.989021
  mse_normalized: 0.041349
  rmse_normalized: 0.203343
  mae_normalized: 0.153564
  correlation: 0.460917
  mse_original: 3.809145
  rmse_original: 1.951703
  rmse_pct_std: 20.334331
  mape_pct: 322548195.516297

Volts:
  signal_variance: 1.007425
  signal_std: 1.003706
  autocorrelation: 0.999956
  mse_normalized: 0.008229
  rmse_normalized: 0.090714
  mae_normalized: 0.061017
  correlation: 0.996335
  mse_original: 0.000649
  rmse_original: 0.025478
  rmse_pct_std: 9.071360
  mape_pct: 0.448550

Temp 1:
  signal_variance: 0.875158
  signal_std: 0.935499
  autocorrelation: 0.997486
  mse_normalized: 0.011348
  rmse_normalized: 0.106525
  mae_normalized: 0.071426
  correlation: 0.993579
  mse_original: 0.633576
  rmse_original: 0.795975
  rmse_pct_std: 10.652490
```

```
  mape_pct: 3.868111

============================================================
COMPRESSION ANALYSIS
============================================================

Model Type: edgecodec

Compression Metrics:
  Average codes per window: 6.00
  Bits per code: 9.6
  Average bits per window: 57.5
  Average bits per timestep: 0.449
  Compression ratio: 213.7x

============================================================
DOWNSTREAM TASK UTILITY
============================================================

Task: Battery Anomaly Detection
Task Type: classification
Samples: 28274

Original Signal Performance:
  accuracy: 0.866136
  precision: 0.125000
  recall: 0.384279
  f1: 0.188639
  auc: 0.735684

Reconstructed Signal Performance:
  accuracy: 0.783908
  precision: 0.080304
  recall: 0.414847
  f1: 0.134561
  auc: 0.684068

Utility Retention: 71.34%
```

Smart Home

Seed 42

```
============================================================
DECODER LOSS ANALYSIS
============================================================
```

```
Normalized MSE (summed): 2.915296
Normalized MSE (meaned): 0.416471
Correlation (meaned): 0.705981

Per-Channel Metrics:
------------------------------------------------------------

lights:
  signal_variance: 0.192364
  signal_std: 0.438593
  autocorrelation: 0.999889
  mse_normalized: 0.087368
  rmse_normalized: 0.295580
  mae_normalized: 0.237220
  correlation: 0.751966
  mse_original: 6.056551
  rmse_original: 2.461006
  rmse_pct_std: 29.557995
  mape_pct: 117.834689

Press_mm_hg:
  signal_variance: 1.466955
  signal_std: 1.211179
  autocorrelation: 0.998368
  mse_normalized: 0.327576
  rmse_normalized: 0.572343
  mae_normalized: 0.460378
  correlation: 0.892436
  mse_original: 19.824161
  rmse_original: 4.452433
  rmse_pct_std: 57.234292
  mape_pct: 0.477106

T_out:
  signal_variance: 0.928853
  signal_std: 0.963770
  autocorrelation: 0.999245
  mse_normalized: 0.539212
  rmse_normalized: 0.734311
  mae_normalized: 0.591283
  correlation: 0.901798
  mse_original: 9.171281
  rmse_original: 3.028412
  rmse_pct_std: 73.431080
  mape_pct: 17.751750
```

```
RH_out:
  signal_variance: 0.972808
  signal_std: 0.986310
  autocorrelation: 0.999397
  mse_normalized: 0.588244
  rmse_normalized: 0.766971
  mae_normalized: 0.620848
  correlation: 0.872877
  mse_original: 99.127585
  rmse_original: 9.956284
  rmse_pct_std: 76.697071
  mape_pct: 7.456402

Windspeed:
  signal_variance: 0.684800
  signal_std: 0.827527
  autocorrelation: 0.984950
  mse_normalized: 0.444197
  rmse_normalized: 0.666481
  mae_normalized: 0.536756
  correlation: 0.605419
  mse_original: 2.884425
  rmse_original: 1.698359
  rmse_pct_std: 66.648109
  mape_pct: 60.426015

Visibility:
  signal_variance: 0.479381
  signal_std: 0.692374
  autocorrelation: 0.996119
  mse_normalized: 0.374615
  rmse_normalized: 0.612058
  mae_normalized: 0.514029
  correlation: 0.602895
  mse_original: 57.339903
  rmse_original: 7.572312
  rmse_pct_std: 61.205797
  mape_pct: 21.459644

Tdewpoint:
  signal_variance: 0.532463
  signal_std: 0.729701
  autocorrelation: 0.774134
  mse_normalized: 0.554084
  rmse_normalized: 0.744368
```

```
  mae_normalized: 0.459025
  correlation: 0.314474
  mse_original: 6.801622
  rmse_original: 2.607992
  rmse_pct_std: 74.436816
  mape_pct: 116.181539

============================================================
COMPRESSION ANALYSIS
============================================================

Model Type: edgecodec

Compression Metrics:
  Average codes per window: 14.00
  Bits per code: 9.6
  Average bits per window: 134.2
  Average bits per timestep: 0.932
  Compression ratio: 240.4x

============================================================
DOWNSTREAM TASK UTILITY
============================================================

Task: Appliance Energy Sequence Prediction
Task Type: regression
Samples: 1831

Original Signal Performance:
  mse: 3424.935426
  mae: 27.434254
  r2: 0.539275
  rmse: 58.522948
  per_timestep_r2_mean: 0.539275
  per_timestep_r2_min: 0.283651
  per_timestep_r2_max: 0.683567
  per_timestep_corr_mean: 0.742444
  per_timestep_corr_min: 0.558071
  target_sequence_length: 144.000000

Reconstructed Signal Performance:
  mse: 3894.811785
  mae: 29.592295
  r2: 0.476322
  rmse: 62.408427
  per_timestep_r2_mean: 0.476322
```

```
  per_timestep_r2_min: 0.233874
  per_timestep_r2_max: 0.605794
  per_timestep_corr_mean: 0.697862
  per_timestep_corr_min: 0.517696
  target_sequence_length: 144.000000

Utility Retention: 88.33%
```

Seed 100

```
============================================================
DECODER LOSS ANALYSIS
============================================================

Normalized MSE (summed): 3.920038

Per-Channel Metrics:
------------------------------------------------------------

lights:
  signal_variance: 0.192364
  signal_std: 0.438593
  autocorrelation: 0.999889
  mse_normalized: 0.117714
  rmse_normalized: 0.343095
  mae_normalized: 0.250767
  correlation: 0.661845
  mse_original: 8.160284
  rmse_original: 2.856621
  rmse_pct_std: 34.309543
  mape_pct: 221.527156

Press_mm_hg:
  signal_variance: 1.466955
  signal_std: 1.211179
  autocorrelation: 0.998368
  mse_normalized: 0.333277
  rmse_normalized: 0.577302
  mae_normalized: 0.464040
  correlation: 0.913980
  mse_original: 20.169161
  rmse_original: 4.491009
  rmse_pct_std: 57.730169
  mape_pct: 0.481682

T_out:
```

```
  signal_variance: 0.928853
  signal_std: 0.963770
  autocorrelation: 0.999245
  mse_normalized: 0.834560
  rmse_normalized: 0.913543
  mae_normalized: 0.767469
  correlation: 0.884189
  mse_original: 14.194755
  rmse_original: 3.767593
  rmse_pct_std: 91.354271
  mape_pct: 23.015442

RH_out:
  signal_variance: 0.972808
  signal_std: 0.986310
  autocorrelation: 0.999397
  mse_normalized: 0.913381
  rmse_normalized: 0.955710
  mae_normalized: 0.778982
  correlation: 0.839455
  mse_original: 153.917827
  rmse_original: 12.406362
  rmse_pct_std: 95.570966
  mape_pct: 9.264668

Windspeed:
  signal_variance: 0.684800
  signal_std: 0.827527
  autocorrelation: 0.984950
  mse_normalized: 0.549037
  rmse_normalized: 0.740970
  mae_normalized: 0.559495
  correlation: 0.501478
  mse_original: 3.565211
  rmse_original: 1.888177
  rmse_pct_std: 74.097039
  mape_pct: 72.546405

Visibility:
  signal_variance: 0.479381
  signal_std: 0.692374
  autocorrelation: 0.996119
  mse_normalized: 0.581787
  rmse_normalized: 0.762750
  mae_normalized: 0.644721
  correlation: 0.380029
```

```
  mse_original: 89.050437
  rmse_original: 9.436654
  rmse_pct_std: 76.274981
  mape_pct: 27.585454

Tdewpoint:
  signal_variance: 0.532463
  signal_std: 0.729701
  autocorrelation: 0.774134
  mse_normalized: 0.590281
  rmse_normalized: 0.768297
  mae_normalized: 0.542656
  correlation: 0.004295
  mse_original: 7.245951
  rmse_original: 2.691830
  rmse_pct_std: 76.829716
  mape_pct: 133.422988

============================================================
COMPRESSION ANALYSIS
============================================================

Model Type: edgecodec

Compression Metrics:
  Average codes per window: 14.00
  Bits per code: 9.6
  Average bits per window: 134.2
  Average bits per timestep: 0.932
  Compression ratio: 240.4x

============================================================
DOWNSTREAM TASK UTILITY
============================================================

Task: Appliance Energy Sequence Prediction
Task Type: regression
Samples: 1831

Original Signal Performance:
  mse: 3424.935426
  mae: 27.434254
  r2: 0.539275
  rmse: 58.522948
  per_timestep_r2_mean: 0.539275
  per_timestep_r2_min: 0.283651
```

```
  per_timestep_r2_max: 0.683567
  per_timestep_corr_mean: 0.742444
  per_timestep_corr_min: 0.558071
  target_sequence_length: 144.000000

Reconstructed Signal Performance:
  mse: 3833.373828
  mae: 29.485584
  r2: 0.484305
  rmse: 61.914246
  per_timestep_r2_mean: 0.484305
  per_timestep_r2_min: 0.193714
  per_timestep_r2_max: 0.672654
  per_timestep_corr_mean: 0.703249
  per_timestep_corr_min: 0.503393
  target_sequence_length: 144.000000

Utility Retention: 89.81%
```

Seed 200

```
============================================================
DECODER LOSS ANALYSIS
============================================================

Normalized MSE (summed): 3.456326

Per-Channel Metrics:
------------------------------------------------------------

lights:
  signal_variance: 0.192364
  signal_std: 0.438593
  autocorrelation: 0.999889
  mse_normalized: 0.169660
  rmse_normalized: 0.411898
  mae_normalized: 0.347094
  correlation: 0.634413
  mse_original: 11.761260
  rmse_original: 3.429469
  rmse_pct_std: 41.189756
  mape_pct: 159.131325

Press_mm_hg:
  signal_variance: 1.466955
  signal_std: 1.211179
```

```
  autocorrelation: 0.998368
  mse_normalized: 0.401867
  rmse_normalized: 0.633930
  mae_normalized: 0.528863
  correlation: 0.911047
  mse_original: 24.320034
  rmse_original: 4.931535
  rmse_pct_std: 63.392956
  mape_pct: 0.549089

T_out:
  signal_variance: 0.928853
  signal_std: 0.963770
  autocorrelation: 0.999245
  mse_normalized: 0.517396
  rmse_normalized: 0.719303
  mae_normalized: 0.599448
  correlation: 0.934587
  mse_original: 8.800222
  rmse_original: 2.966517
  rmse_pct_std: 71.930277
  mape_pct: 17.935747

RH_out:
  signal_variance: 0.972808
  signal_std: 0.986310
  autocorrelation: 0.999397
  mse_normalized: 0.585227
  rmse_normalized: 0.765001
  mae_normalized: 0.639446
  correlation: 0.918012
  mse_original: 98.619092
  rmse_original: 9.930715
  rmse_pct_std: 76.500102
  mape_pct: 7.663384

Windspeed:
  signal_variance: 0.684800
  signal_std: 0.827527
  autocorrelation: 0.984950
  mse_normalized: 0.684107
  rmse_normalized: 0.827107
  mae_normalized: 0.614823
  correlation: 0.169944
  mse_original: 4.442295
  rmse_original: 2.107675
```

```
  rmse_pct_std: 82.710738
  mape_pct: 81.250820

Visibility:
  signal_variance: 0.479381
  signal_std: 0.692374
  autocorrelation: 0.996119
  mse_normalized: 0.524390
  rmse_normalized: 0.724148
  mae_normalized: 0.601978
  correlation: 0.651912
  mse_original: 80.265043
  rmse_original: 8.959076
  rmse_pct_std: 72.414795
  mape_pct: 26.596621

Tdewpoint:
  signal_variance: 0.532463
  signal_std: 0.729701
  autocorrelation: 0.774134
  mse_normalized: 0.573680
  rmse_normalized: 0.757417
  mae_normalized: 0.565985
  correlation: -0.005561
  mse_original: 7.042172
  rmse_original: 2.653709
  rmse_pct_std: 75.741665
  mape_pct: 143.624932

============================================================
COMPRESSION ANALYSIS
============================================================

Model Type: edgecodec

Compression Metrics:
  Average codes per window: 14.00
  Bits per code: 9.6
  Average bits per window: 134.2
  Average bits per timestep: 0.932
  Compression ratio: 240.4x

============================================================
DOWNSTREAM TASK UTILITY
============================================================
```

```
Task: Appliance Energy Sequence Prediction
Task Type: regression
Samples: 1831

Original Signal Performance:
  mse: 3424.935426
  mae: 27.434254
  r2: 0.539275
  rmse: 58.522948
  per_timestep_r2_mean: 0.539275
  per_timestep_r2_min: 0.283651
  per_timestep_r2_max: 0.683567
  per_timestep_corr_mean: 0.742444
  per_timestep_corr_min: 0.558071
  target_sequence_length: 144.000000

Reconstructed Signal Performance:
  mse: 3743.682718
  mae: 28.839653
  r2: 0.497435
  rmse: 61.185641
  per_timestep_r2_mean: 0.497435
  per_timestep_r2_min: 0.229899
  per_timestep_r2_max: 0.648925
  per_timestep_corr_mean: 0.712466
  per_timestep_corr_min: 0.525899
  target_sequence_length: 144.000000

Utility Retention: 92.24%
```

TCN-RNN

EMOB

Seed 42

```
============================================================
DECODER LOSS ANALYSIS
============================================================

Normalized MSE (summed): 0.370266
Normalized MSE (meaned): 0.123422
Correlation (meaned): 0.726090

Per-Channel Metrics:
```

```
------------------------------------------------------------

Amps:
  signal_variance: 0.052480
  signal_std: 0.229085
  autocorrelation: 0.989021
  mse_normalized: 0.047846
  rmse_normalized: 0.218736
  mae_normalized: 0.171020
  correlation: 0.303573
  mse_original: 4.407678
  rmse_original: 2.099447
  rmse_pct_std: 21.873641
  mape_pct: 686933040.197145

Volts:
  signal_variance: 1.007425
  signal_std: 1.003706
  autocorrelation: 0.999956
  mse_normalized: 0.182852
  rmse_normalized: 0.427613
  mae_normalized: 0.248458
  correlation: 0.910052
  mse_original: 0.014424
  rmse_original: 0.120101
  rmse_pct_std: 42.761253
  mape_pct: 1.801796

Temp 1:
  signal_variance: 0.875158
  signal_std: 0.935499
  autocorrelation: 0.997486
  mse_normalized: 0.139568
  rmse_normalized: 0.373588
  mae_normalized: 0.311113
  correlation: 0.964646
  mse_original: 7.792603
  rmse_original: 2.791524
  rmse_pct_std: 37.358814
  mape_pct: 12.823147

============================================================
COMPRESSION ANALYSIS
============================================================

Model Type: baseline
```

```
Baseline Compression (continuous embedding):
  Embedding dimension: 1
  Uncompressed bits per window: 12288
  Compressed bits per window: 4096
  Compression ratio: 3.0x
```

Smart Home

Seed 42

```
============================================================
DECODER LOSS ANALYSIS
============================================================

Normalized MSE (summed): 0.133435
Normalized MSE (meaned): 0.066717
Correlation (meaned): 0.952112

Per-Channel Metrics:
------------------------------------------------------------

Appliances:
  signal_variance: 0.788840
  signal_std: 0.888167
  autocorrelation: 0.741262
  mse_normalized: 0.063541
  rmse_normalized: 0.252074
  mae_normalized: 0.185539
  correlation: 0.964537
  mse_original: 710.454994
  rmse_original: 26.654362
  rmse_pct_std: 25.207365
  mape_pct: 23.245321

Windspeed:
  signal_variance: 0.479381
  signal_std: 0.692374
  autocorrelation: 0.996119
  mse_normalized: 0.069894
  rmse_normalized: 0.264375
  mae_normalized: 0.189048
  correlation: 0.939688
  mse_original: 0.453862
  rmse_original: 0.673693
  rmse_pct_std: 26.437479
  mape_pct: 1389198.062052
```

```
============================================================
COMPRESSION ANALYSIS
============================================================

Model Type: baseline

Baseline Compression (continuous embedding):
  Embedding dimension: 1
  Uncompressed bits per window: 9216
  Compressed bits per window: 4608
  Compression ratio: 2.0x
```

Seed 42

```
============================================================
DECODER LOSS ANALYSIS
============================================================

Normalized MSE (summed): 5.395185
Normalized MSE (meaned): 0.770741
Correlation (meaned): 0.651379

Per-Channel Metrics:
------------------------------------------------------------

lights:
  signal_variance: 0.192364
  signal_std: 0.438593
  autocorrelation: 0.999889
  mse_normalized: 0.691451
  rmse_normalized: 0.831535
  mae_normalized: 0.739601
  correlation: 0.481432
  mse_original: 47.933262
  rmse_original: 6.923385
  rmse_pct_std: 83.153548
  mape_pct: 183.876285

Press_mm_hg:
  signal_variance: 1.466955
  signal_std: 1.211179
  autocorrelation: 0.998368
  mse_normalized: 0.413481
  rmse_normalized: 0.643025
```

```
  mae_normalized: 0.490716
  correlation: 0.848220
  mse_original: 25.022917
  rmse_original: 5.002291
  rmse_pct_std: 64.302503
  mape_pct: 0.508225

T_out:
  signal_variance: 0.928853
  signal_std: 0.963770
  autocorrelation: 0.999245
  mse_normalized: 0.554971
  rmse_normalized: 0.744964
  mae_normalized: 0.565004
  correlation: 0.831086
  mse_original: 9.439311
  rmse_original: 3.072346
  rmse_pct_std: 74.496360
  mape_pct: 16.633342

RH_out:
  signal_variance: 0.972808
  signal_std: 0.986310
  autocorrelation: 0.999397
  mse_normalized: 0.925466
  rmse_normalized: 0.962012
  mae_normalized: 0.766366
  correlation: 0.592619
  mse_original: 155.954362
  rmse_original: 12.488169
  rmse_pct_std: 96.201153
  mape_pct: 9.277289

Windspeed:
  signal_variance: 0.684800
  signal_std: 0.827527
  autocorrelation: 0.984950
  mse_normalized: 0.638486
  rmse_normalized: 0.799053
  mae_normalized: 0.580331
  correlation: 0.449330
  mse_original: 4.146057
  rmse_original: 2.036187
  rmse_pct_std: 79.905345
  mape_pct: 82.591382
```

```
Visibility:
  signal_variance: 0.479381
  signal_std: 0.692374
  autocorrelation: 0.996119
  mse_normalized: 2.132777
  rmse_normalized: 1.460403
  mae_normalized: 1.318764
  correlation: 0.392280
  mse_original: 326.450519
  rmse_original: 18.067942
  rmse_pct_std: 146.040316
  mape_pct: 56.341707

Tdewpoint:
  signal_variance: 0.532463
  signal_std: 0.729701
  autocorrelation: 0.774134
  mse_normalized: 0.038552
  rmse_normalized: 0.196347
  mae_normalized: 0.131337
  correlation: 0.964688
  mse_original: 0.473244
  rmse_original: 0.687927
  rmse_pct_std: 19.634696
  mape_pct: 34.713490

============================================================
COMPRESSION ANALYSIS
============================================================

Model Type: baseline

Baseline Compression (continuous embedding):
  Embedding dimension: 1
  Uncompressed bits per window: 32256
  Compressed bits per window: 4608
  Compression ratio: 7.0x
```